

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

Interconnects (Part 4)

Instructor: Prof. Joshua San Miguel

Some slides adopted from Natalie Enright Jerger

Last Lecture

- Mesh
- Indirect topologies
- Butterfly
- Deterministic routing
- Oblivious routing

This Lecture

- Adaptive routing
- Routing deadlocks
- Circuit-switching
- Packet-based flow control

Adaptive Routing

Use network state to make routing decisions

- Often use buffer occupancies as proxy for network state (i.e., congestion)
- Coupled with flow control implementation
- Can be minimal or non-minimal

Local Information:

- Use information about neighbors only
- Cheaper to obtain local information; flow control already employs mechanisms to track next-hop buffer occupancy
- Can lead to suboptimal choices

Global Information:

- Use information about all nodes; more holistic view of network state
- Routers must send explicit status information by piggybacking packet payloads or using a side network
- Can take longer to detect changes in network state

Adaptive Routing – Minimal

Only route through shortest paths:

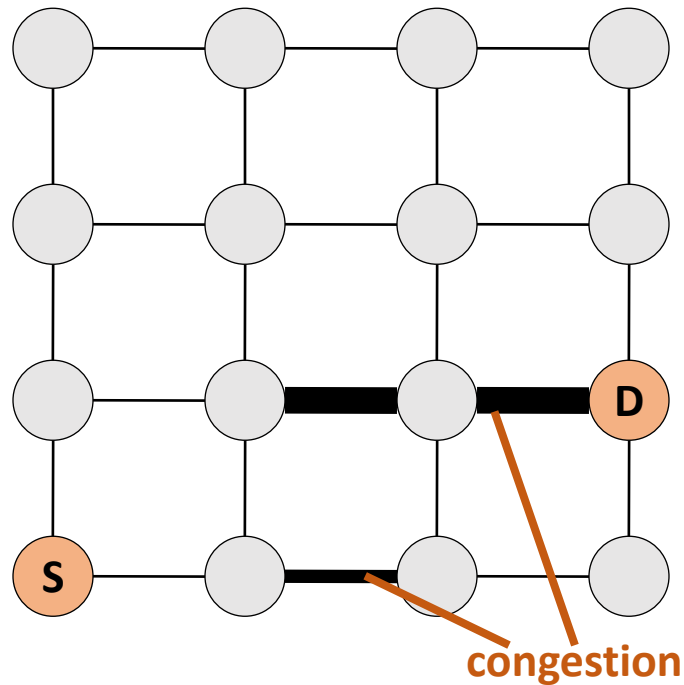
- Relies on path diversity
- Typically uses local information

Adaptive Routing – Minimal

Only route through shortest paths:

- Relies on path diversity
- Typically uses local information

e.g., Assume local information:

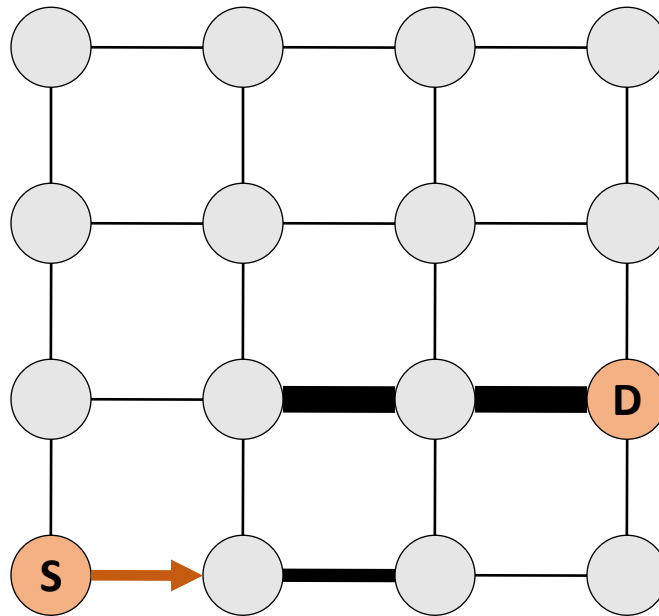


Adaptive Routing – Minimal

Only route through shortest paths:

- Relies on path diversity
- Typically uses local information

e.g., Assume local information:

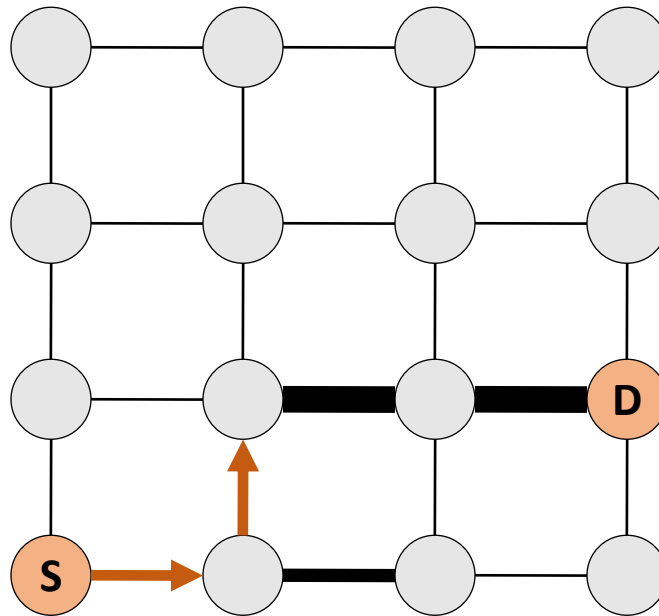


Adaptive Routing – Minimal

Only route through shortest paths:

- Relies on path diversity
- Typically uses local information

e.g., Assume local information:



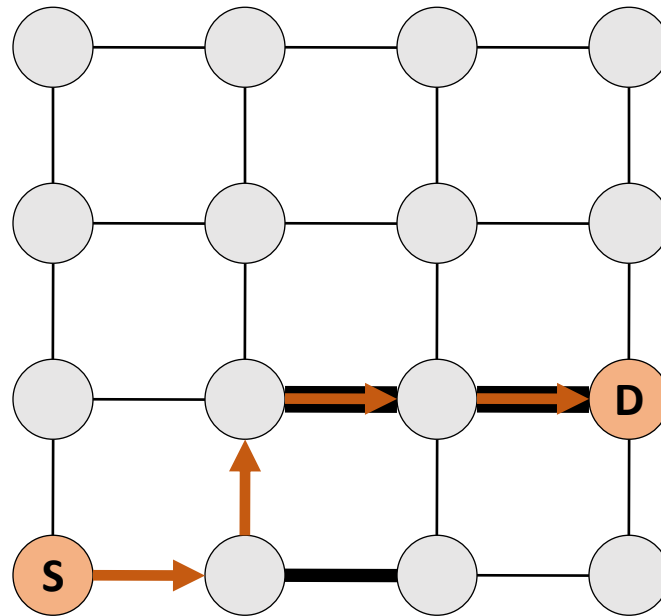
Adaptive Routing – Minimal

Only route through shortest paths:

- Relies on path diversity
- Typically uses local information

e.g., Assume local information:

Ends up taking most congested route



Adaptive Routing – Non-Minimal

Not limited to shortest paths (i.e., fully adaptive)

- Still try to minimize path length
- Necessary mechanisms for fault tolerance

Misrouting: Directing a packet along a non-productive link

- Priority given to productive output
- Some routing algorithms forbid U-turns

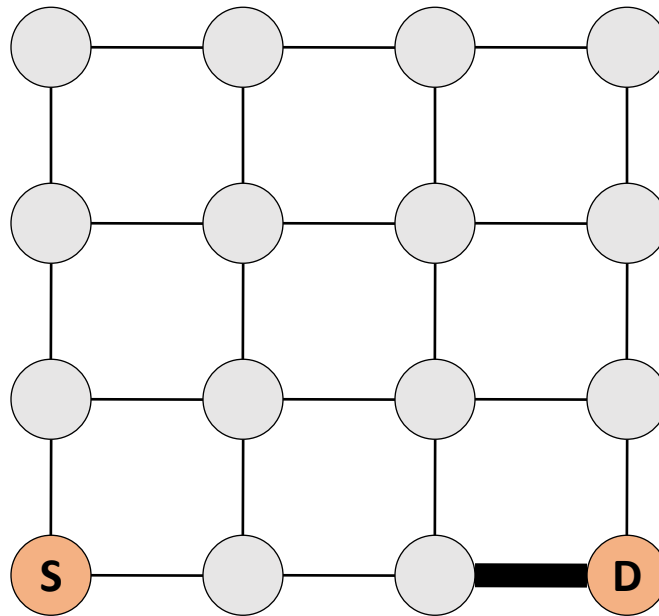
Livelock: Traversing network without ever reaching destination

- Need mechanism to guarantee forward progress; limit the number of misroutings

Adaptive Routing – Non-Minimal

Not limited to shortest paths (i.e., fully adaptive)

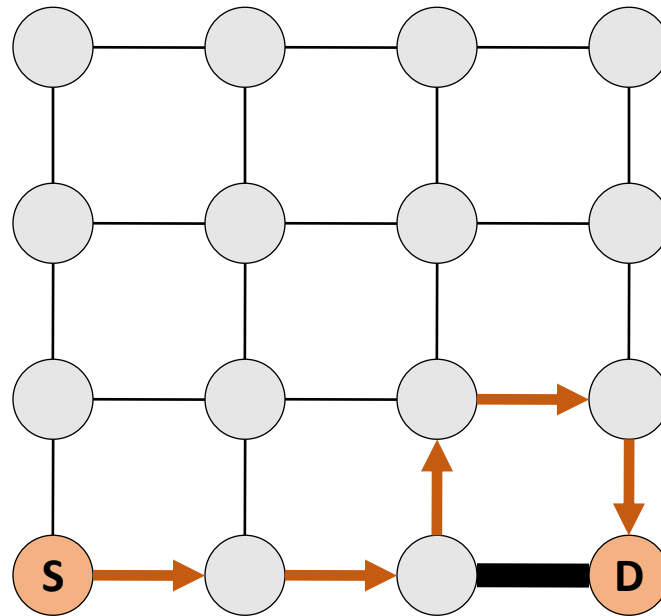
- Still try to minimize path length
- Necessary mechanisms for fault tolerance



Adaptive Routing – Non-Minimal

Not limited to shortest paths (i.e., fully adaptive)

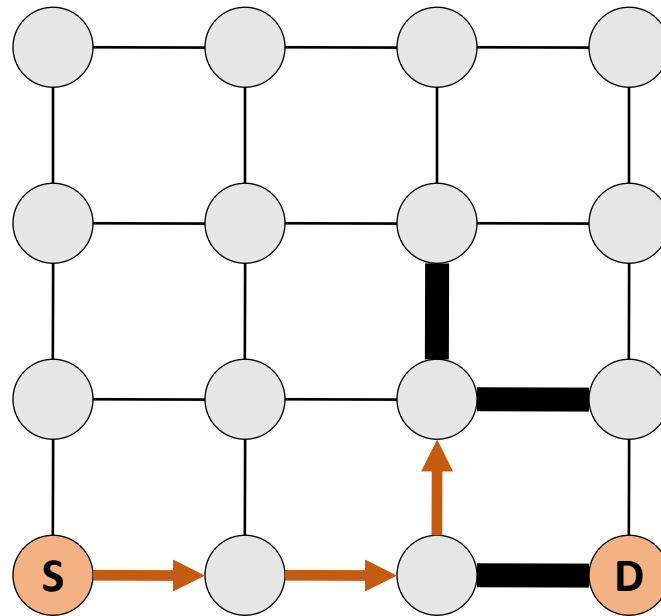
- Still try to minimize path length
- Necessary mechanisms for fault tolerance



Adaptive Routing – Non-Minimal

Not limited to shortest paths (i.e., fully adaptive)

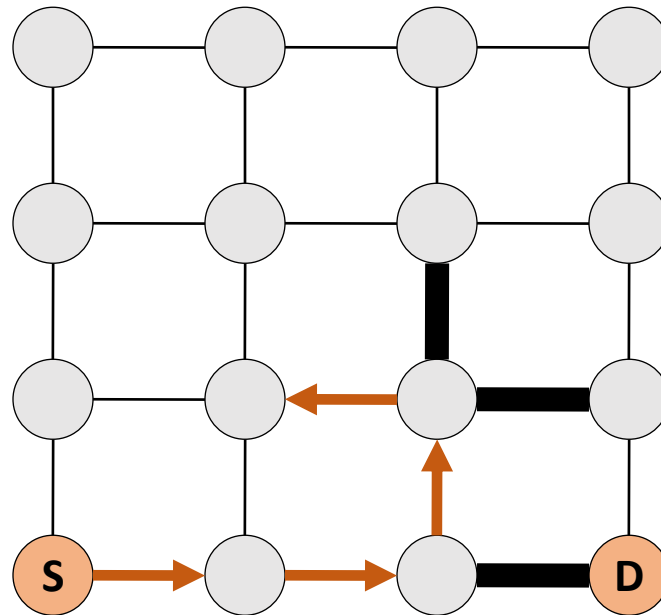
- Still try to minimize path length
- Necessary mechanisms for fault tolerance



Adaptive Routing – Non-Minimal

Not limited to shortest paths (i.e., fully adaptive)

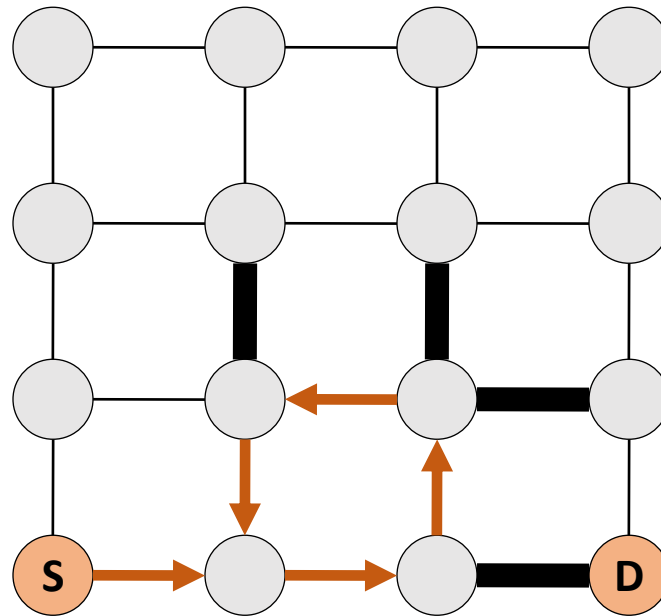
- Still try to minimize path length
- Necessary mechanisms for fault tolerance



Adaptive Routing – Non-Minimal

Not limited to shortest paths (i.e., fully adaptive)

- Still try to minimize path length
- Necessary mechanisms for fault tolerance



Adaptive Routing – Backpressure

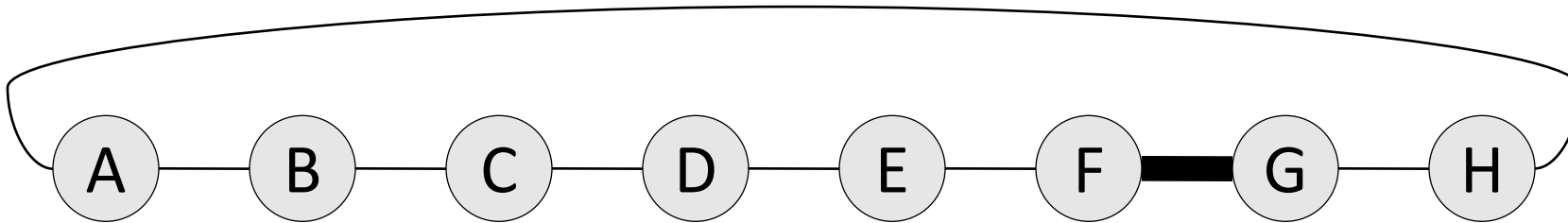
Backpressure: Allows routers to indirectly sense congestion

- When buffer/queue in one router fills up, it will stop receiving packets
- As a result, incoming buffers will also fill up

Adaptive Routing – Backpressure

Backpressure: Allows routers to indirectly sense congestion

- When buffer/queue in one router fills up, it will stop receiving packets
- As a result, incoming buffers will also fill up



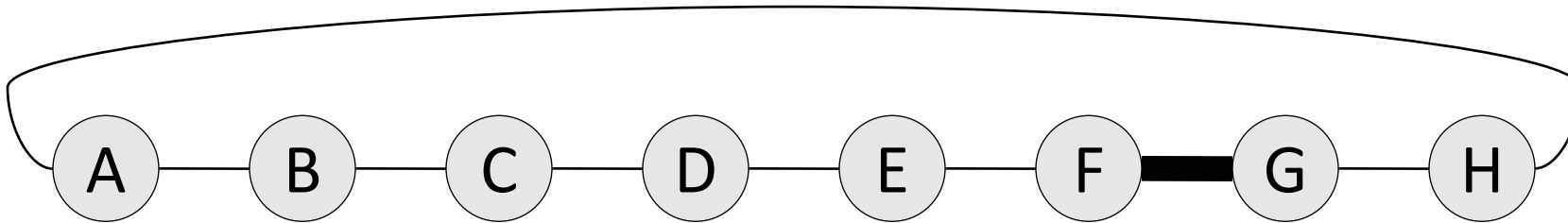
e.g., Router D to G:

Assume local information

Adaptive Routing – Backpressure

Backpressure: Allows routers to indirectly sense congestion

- When buffer/queue in one router fills up, it will stop receiving packets
- As a result, incoming buffers will also fill up



e.g., Router D to G:

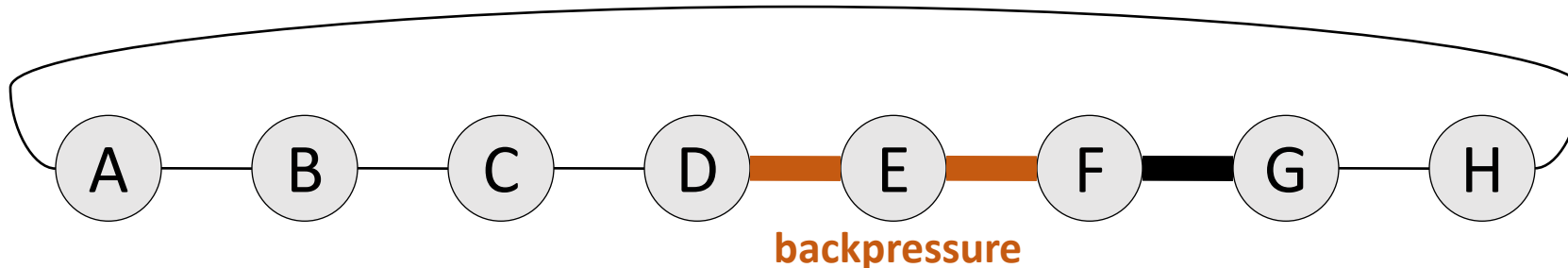
Assume local information

D's packet should traverse counter-clockwise to G

Adaptive Routing – Backpressure

Backpressure: Allows routers to indirectly sense congestion

- When buffer/queue in one router fills up, it will stop receiving packets
- As a result, incoming buffers will also fill up



e.g., Router D to G:

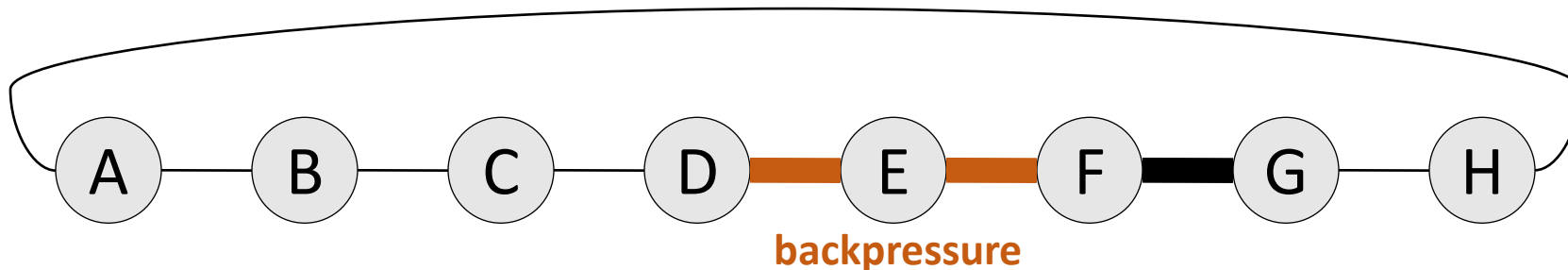
Assume local information

D's packet should traverse counter-clockwise to G

Adaptive Routing – Backpressure

Backpressure: Allows routers to indirectly sense congestion

- When buffer/queue in one router fills up, it will stop receiving packets
- As a result, incoming buffers will also fill up



e.g., Router D to G:

Assume local information

D's packet should traverse counter-clockwise to G

If 4-packet buffer, router D will sense congestion after sending 8 packets

Adaptive Routing – Deadlocks

Option 1:

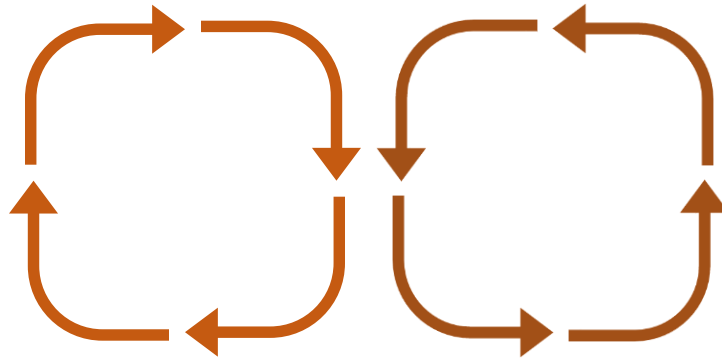
- **Turn Model:** Disallow some turns to prevent deadlocks
- Limits routing flexibility and adaptivity (i.e., less path diversity)

Option 2:

- **Deadlock-Free Flow Control:** Rely on flow control mechanisms
- Incurs buffering overhead and hardware complexity
- e.g., escape virtual channels, bubble flow control (discussed later)

Adaptive Routing – Turn Model

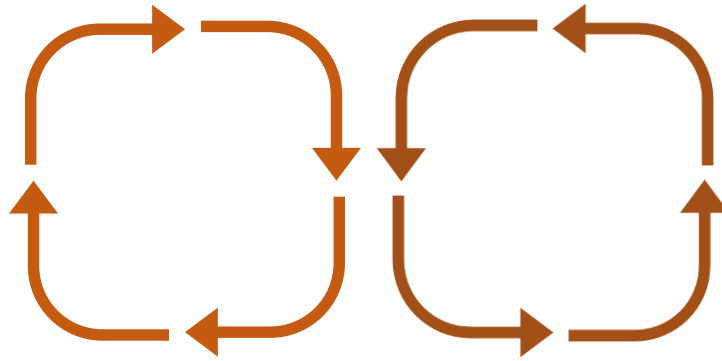
Turn Model: Disallow some turns to prevent deadlocks



Adaptive Routing – Turn Model

Turn Model: Disallow some turns to prevent deadlocks

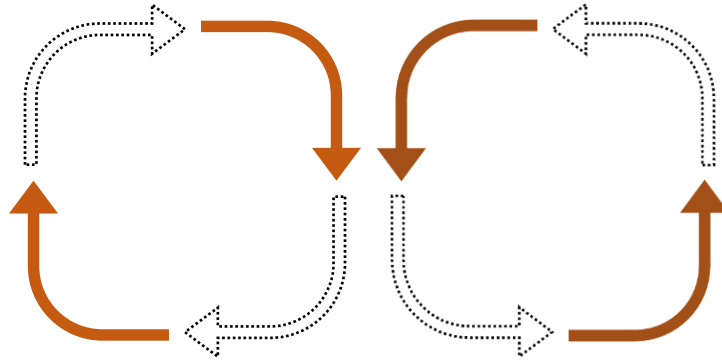
- X-Y DOR eliminates 4 turns; no adaptivity



Adaptive Routing – Turn Model

Turn Model: Disallow some turns to prevent deadlocks

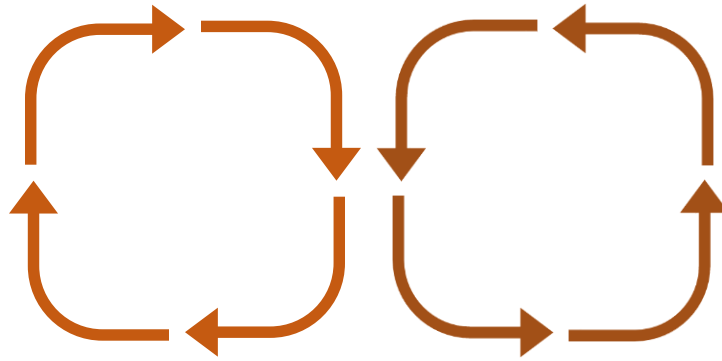
- X-Y DOR eliminates 4 turns; no adaptivity



Adaptive Routing – Turn Model

Turn Model: Disallow some turns to prevent deadlocks

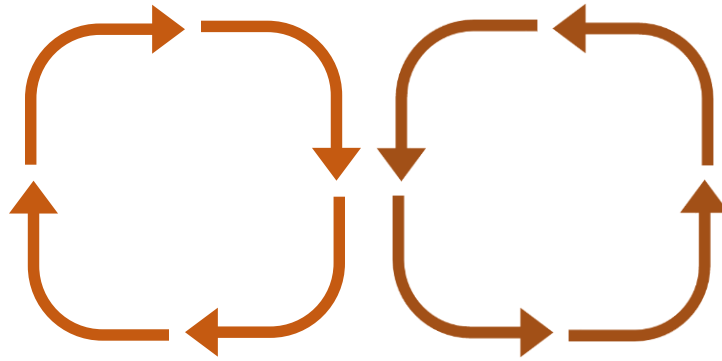
- X-Y DOR eliminates 4 turns; no adaptivity
- Minimum number of turns that need to be eliminated to ensure deadlock-freedom?



Adaptive Routing – Turn Model

Turn Model: Disallow some turns to prevent deadlocks

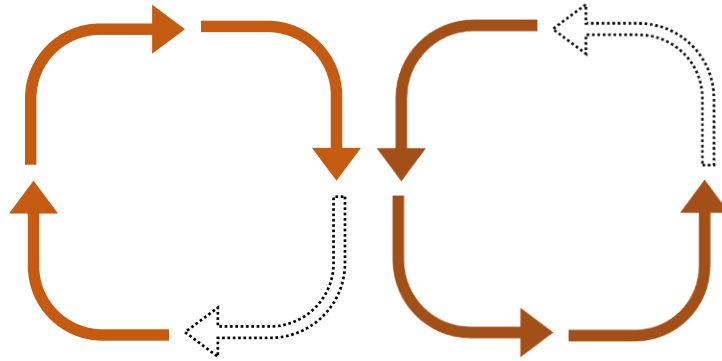
- X-Y DOR eliminates 4 turns; no adaptivity
- Minimum number of turns that need to be eliminated to ensure deadlock-freedom? **2**



Adaptive Routing – Turn Model

Turn Model: Disallow some turns to prevent deadlocks

- X-Y DOR eliminates 4 turns; no adaptivity
- Minimum number of turns that need to be eliminated to ensure deadlock-freedom? **2**



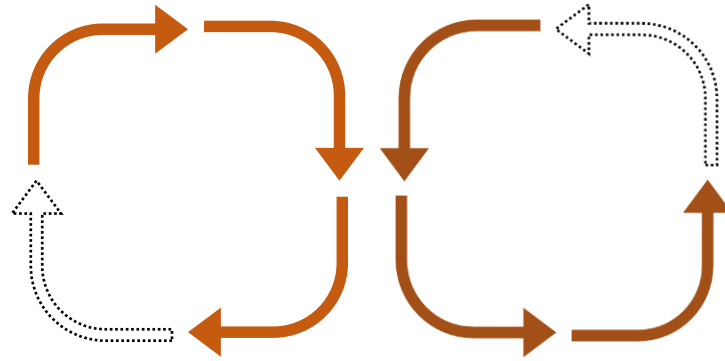
e.g., West First:

Eliminate S to W and N to W

Adaptive Routing – Turn Model

Turn Model: Disallow some turns to prevent deadlocks

- X-Y DOR eliminates 4 turns; no adaptivity
- Minimum number of turns that need to be eliminated to ensure deadlock-freedom? **2**

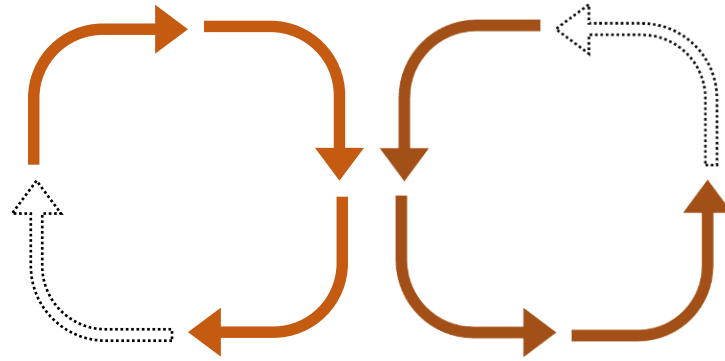


Can we eliminate these turns?

Adaptive Routing – Turn Model

Turn Model: Disallow some turns to prevent deadlocks

- X-Y DOR eliminates 4 turns; no adaptivity
- Minimum number of turns that need to be eliminated to ensure deadlock-freedom? **2**



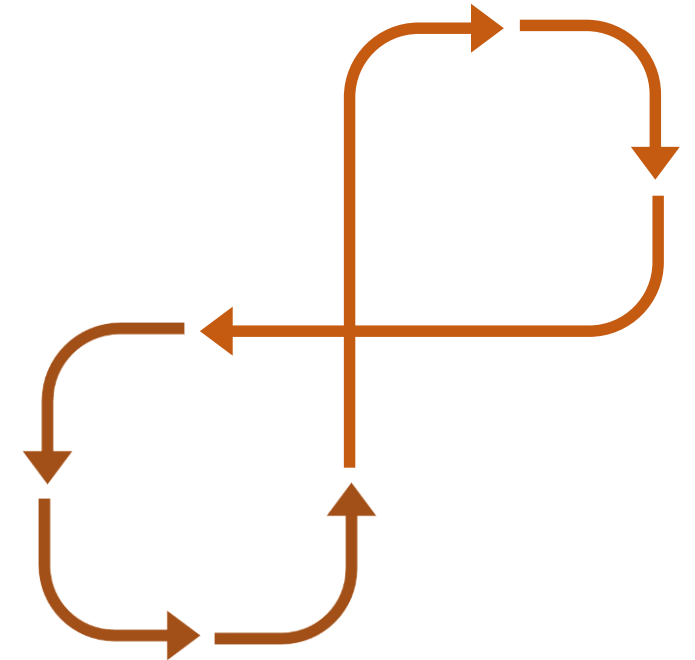
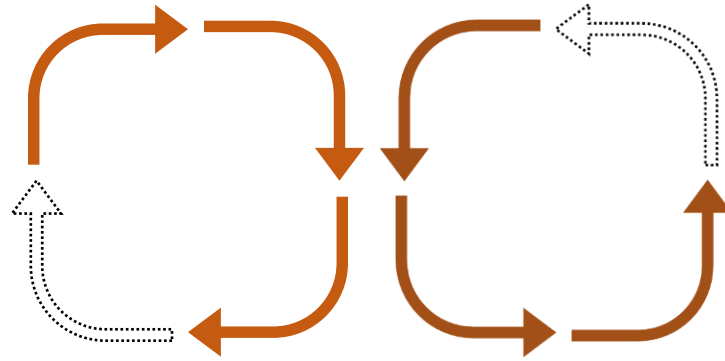
Can we eliminate these turns?

No

Adaptive Routing – Turn Model

Turn Model: Disallow some turns to prevent deadlocks

- X-Y DOR eliminates 4 turns; no adaptivity
- Minimum number of turns that need to be eliminated to ensure deadlock-freedom? **2**



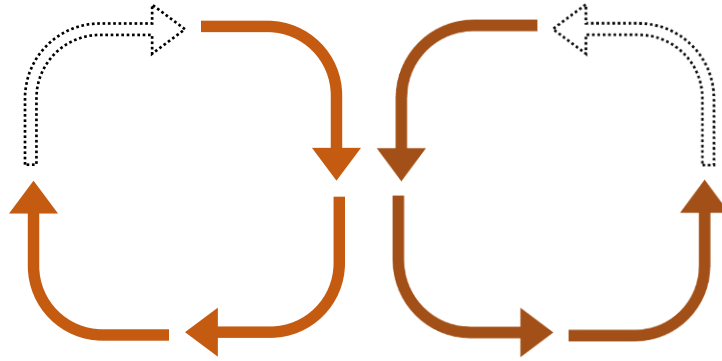
Can we eliminate these turns?

No

Adaptive Routing – Turn Model

Turn Model: Disallow some turns to prevent deadlocks

- X-Y DOR eliminates 4 turns; no adaptivity
- Minimum number of turns that need to be eliminated to ensure deadlock-freedom? **2**

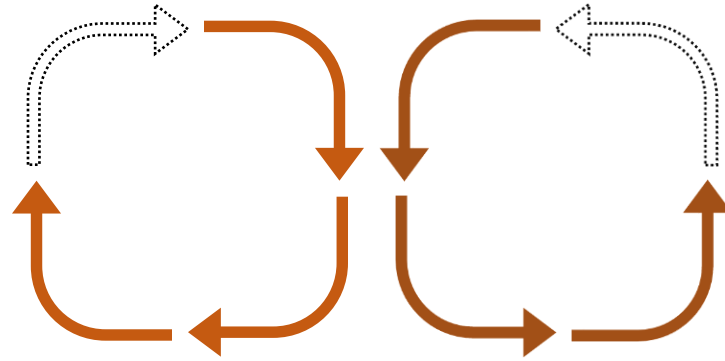


Can we eliminate these turns?

Adaptive Routing – Turn Model

Turn Model: Disallow some turns to prevent deadlocks

- X-Y DOR eliminates 4 turns; no adaptivity
- Minimum number of turns that need to be eliminated to ensure deadlock-freedom? **2**



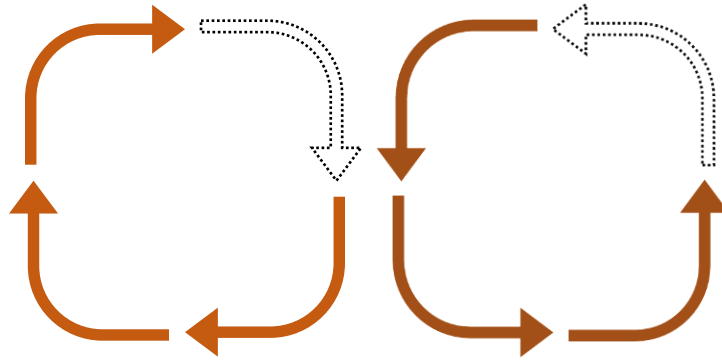
Can we eliminate these turns?

Yes, North Last

Adaptive Routing – Turn Model

Turn Model: Disallow some turns to prevent deadlocks

- X-Y DOR eliminates 4 turns; no adaptivity
- Minimum number of turns that need to be eliminated to ensure deadlock-freedom? **2**

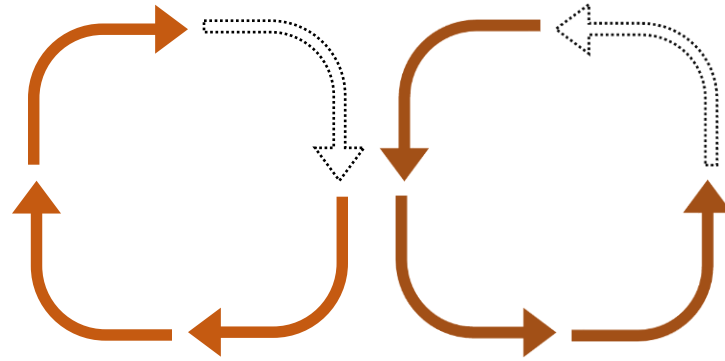


Can we eliminate these turns?

Adaptive Routing – Turn Model

Turn Model: Disallow some turns to prevent deadlocks

- X-Y DOR eliminates 4 turns; no adaptivity
- Minimum number of turns that need to be eliminated to ensure deadlock-freedom? **2**



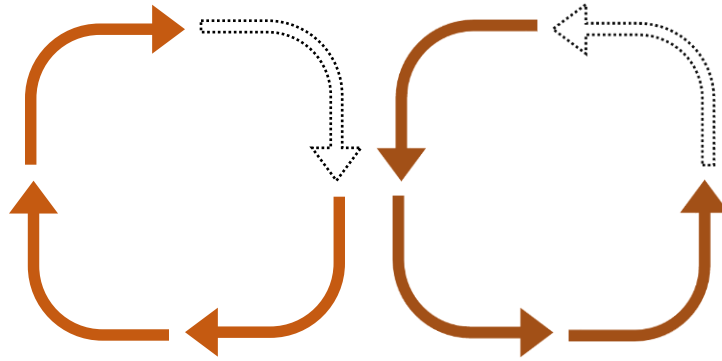
Can we eliminate these turns?

Yes, Negative First

Adaptive Routing – Turn Model

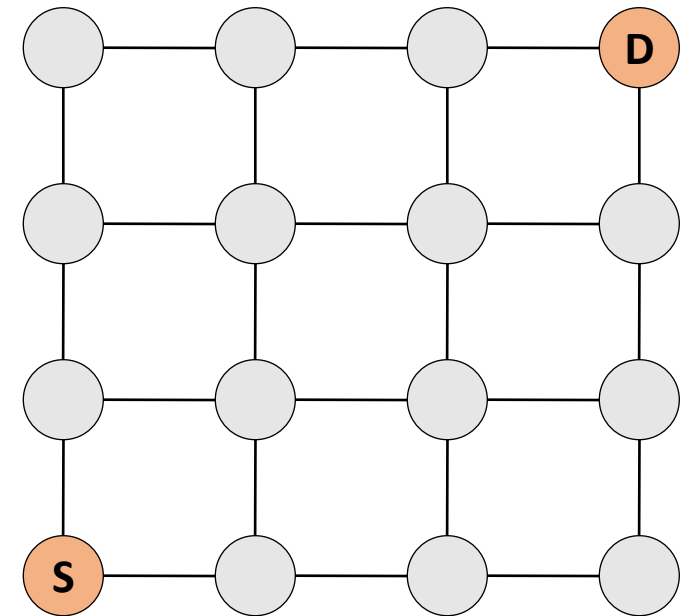
Turn Model: Disallow some turns to prevent deadlocks

- X-Y DOR eliminates 4 turns; no adaptivity
- Minimum number of turns that need to be eliminated to ensure deadlock-freedom? **2**



Can we eliminate these turns?

Yes, Negative First



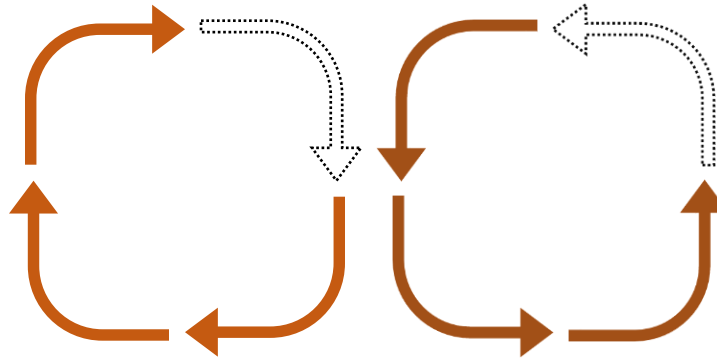
Path diversity?



Adaptive Routing – Turn Model

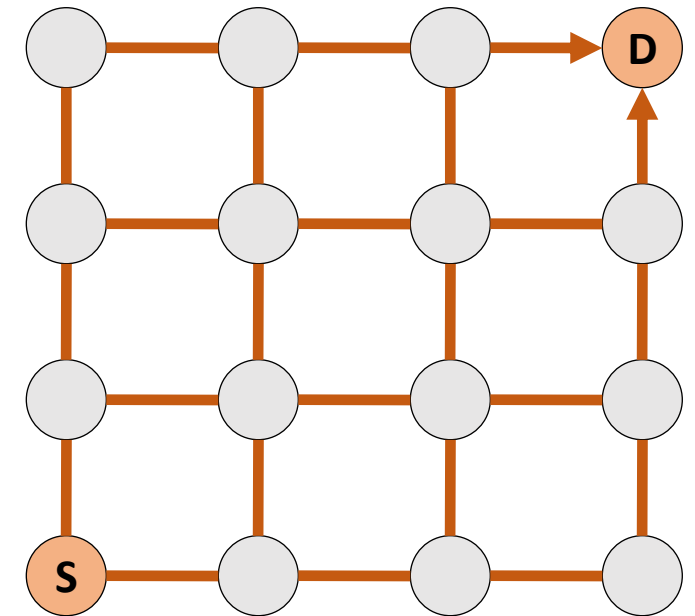
Turn Model: Disallow some turns to prevent deadlocks

- X-Y DOR eliminates 4 turns; no adaptivity
- Minimum number of turns that need to be eliminated to ensure deadlock-freedom? **2**



Can we eliminate these turns?

Yes, Negative First



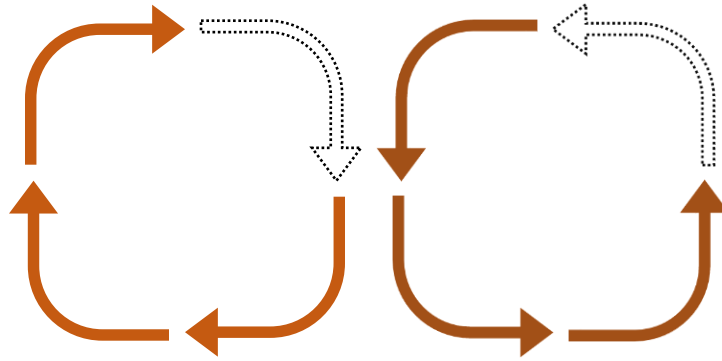
Path diversity? 20 paths



Adaptive Routing – Turn Model

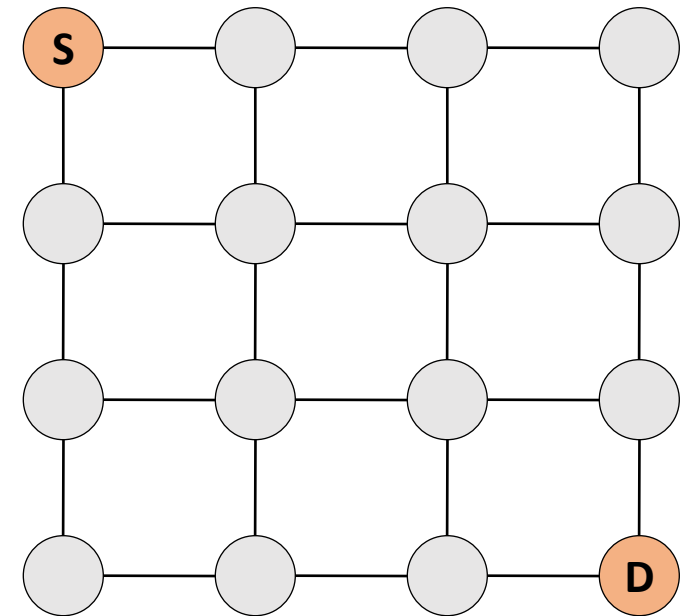
Turn Model: Disallow some turns to prevent deadlocks

- X-Y DOR eliminates 4 turns; no adaptivity
- Minimum number of turns that need to be eliminated to ensure deadlock-freedom? **2**



Can we eliminate these turns?

Yes, Negative First



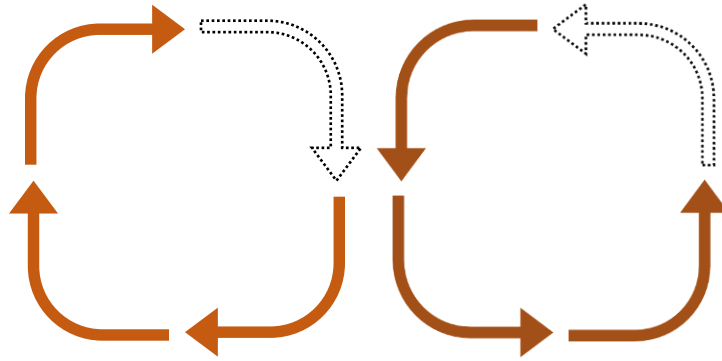
Path diversity?



Adaptive Routing – Turn Model

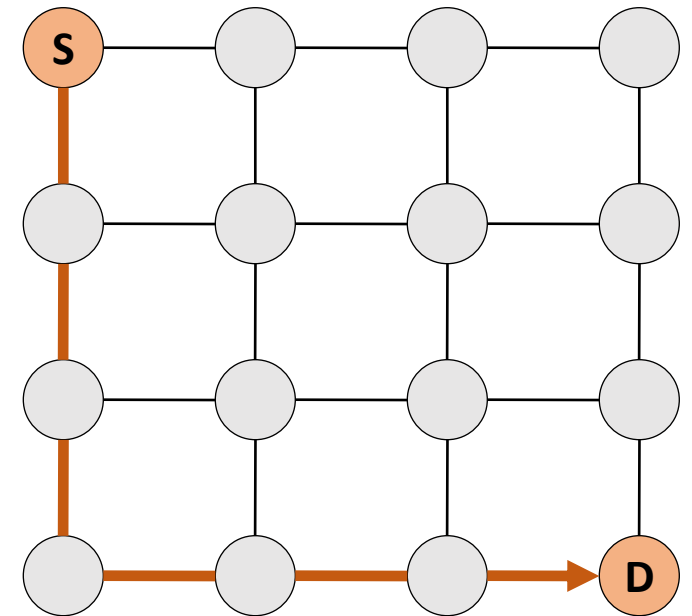
Turn Model: Disallow some turns to prevent deadlocks

- X-Y DOR eliminates 4 turns; no adaptivity
- Minimum number of turns that need to be eliminated to ensure deadlock-freedom? **2**



Can we eliminate these turns?

Yes, Negative First



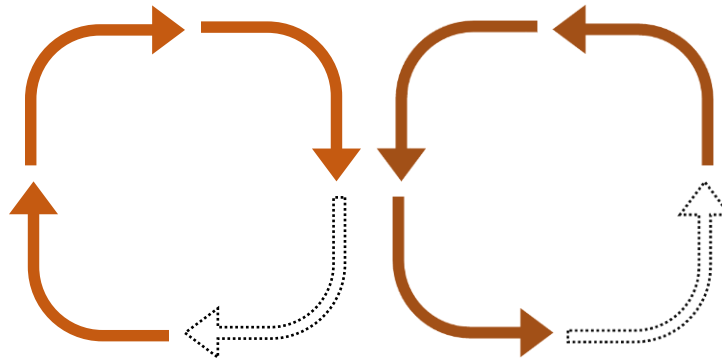
Path diversity? 1 path



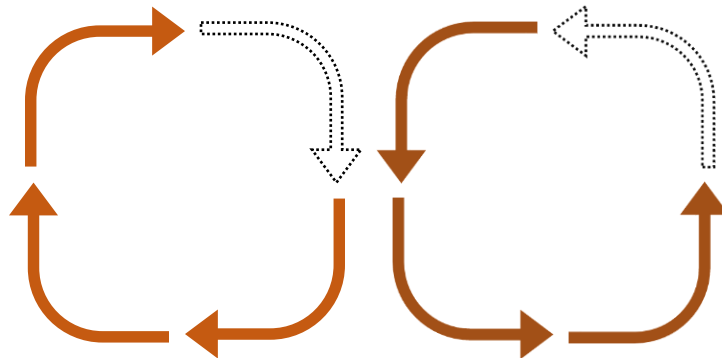
Adaptive Routing – Turn Model

Odd-Even:

- More balanced path diversity; better adaptivity
- Effectively, once a packet starts traversing east, it is never allowed to go west again
- If current router is in even column: eliminate S to W and E to N



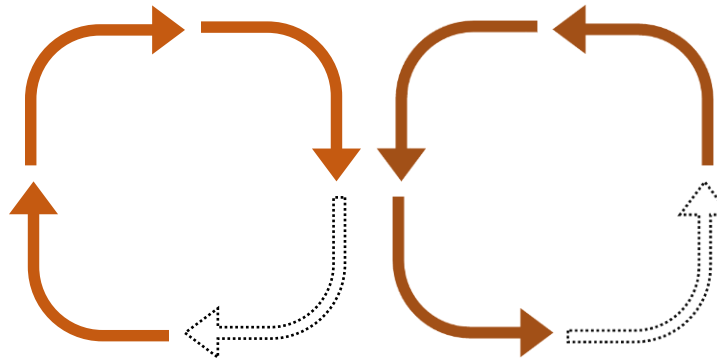
- If current router is in odd column: eliminate E to S and N to W



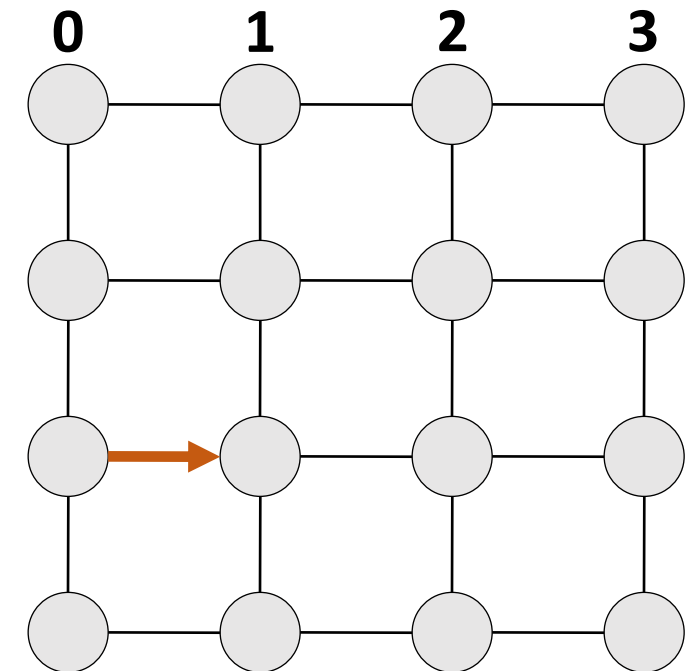
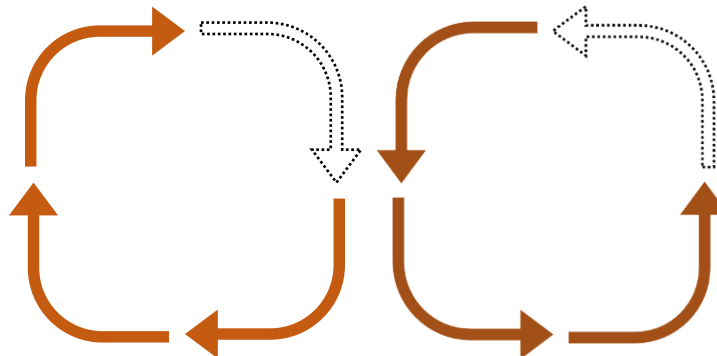
Adaptive Routing – Turn Model

Odd-Even:

- More balanced path diversity; better adaptivity
- Effectively, once a packet starts traversing east, it is never allowed to go west again
- If current router is in even column: eliminate S to W and E to N



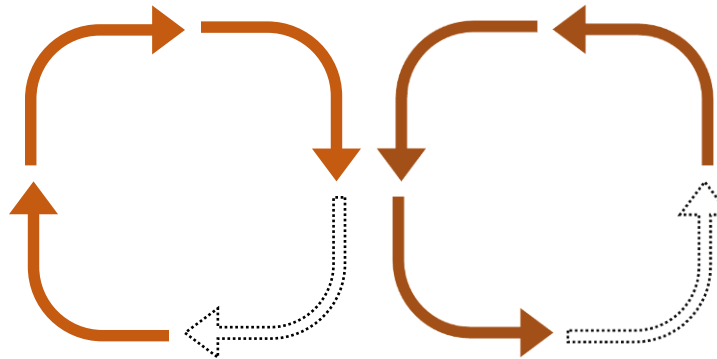
- If current router is in odd column: eliminate E to S and N to W



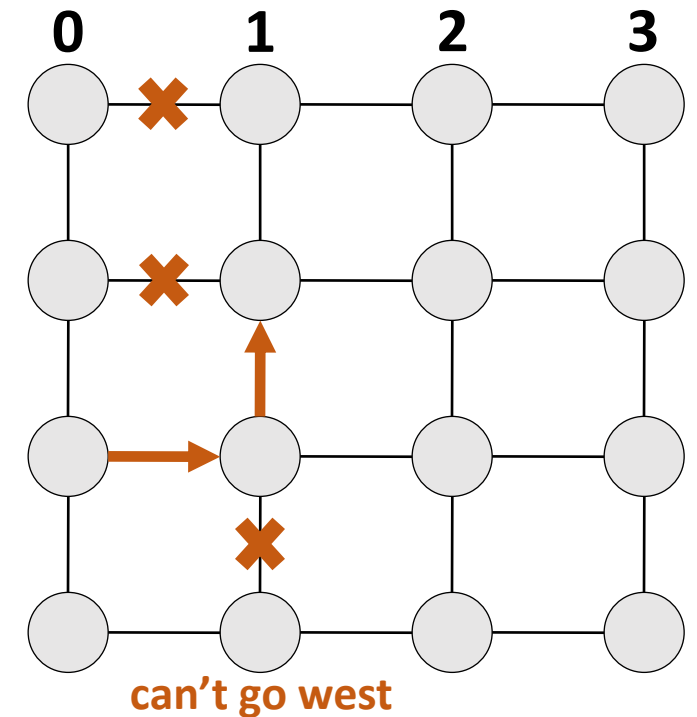
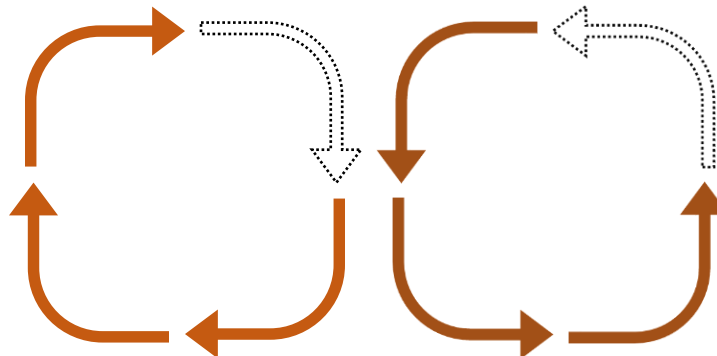
Adaptive Routing – Turn Model

Odd-Even:

- More balanced path diversity; better adaptivity
- Effectively, once a packet starts traversing east, it is never allowed to go west again
- If current router is in even column: eliminate S to W and E to N



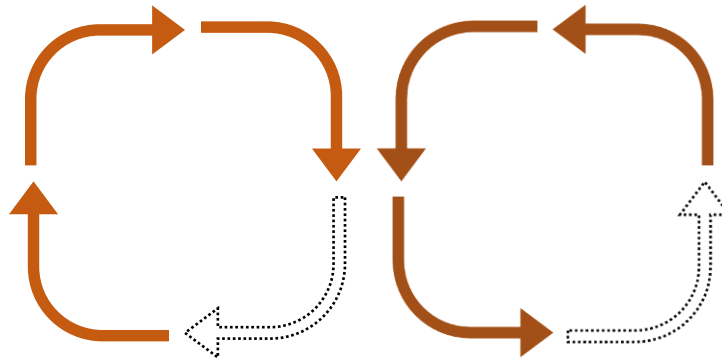
- If current router is in odd column: eliminate E to S and N to W



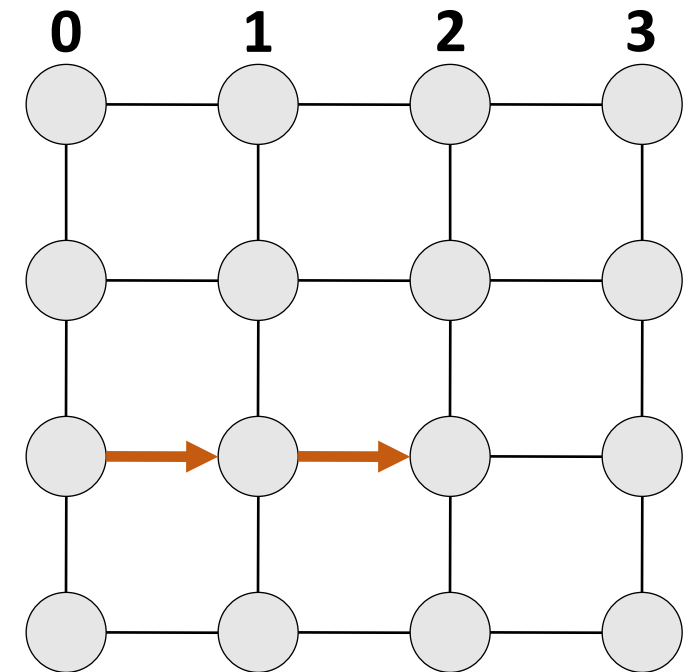
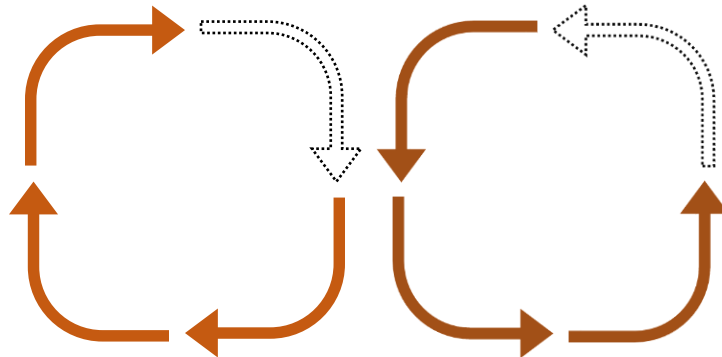
Adaptive Routing – Turn Model

Odd-Even:

- More balanced path diversity; better adaptivity
- Effectively, once a packet starts traversing east, it is never allowed to go west again
- If current router is in even column: eliminate S to W and E to N



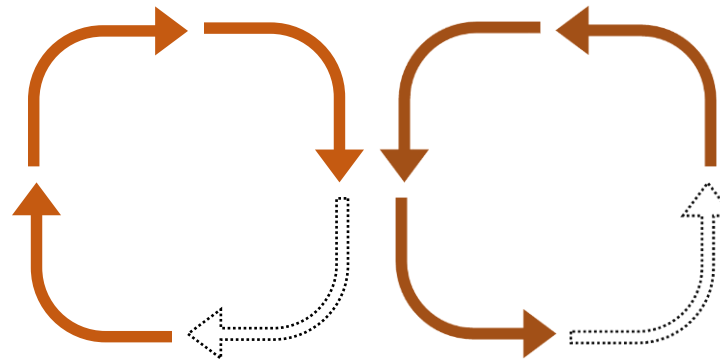
- If current router is in odd column: eliminate E to S and N to W



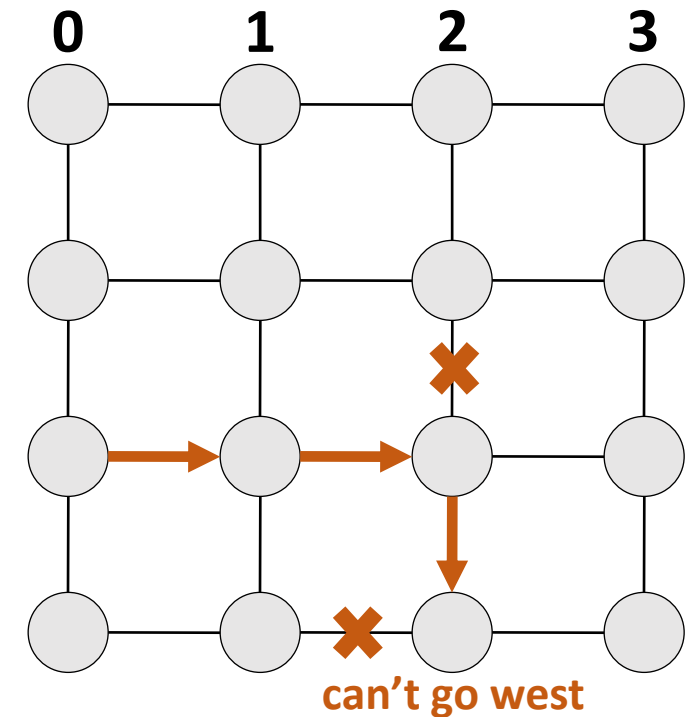
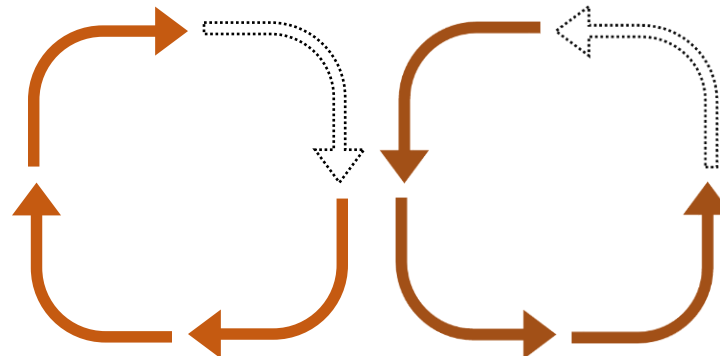
Adaptive Routing – Turn Model

Odd-Even:

- More balanced path diversity; better adaptivity
- Effectively, once a packet starts traversing east, it is never allowed to go west again
- If current router is in even column: eliminate S to W and E to N



- If current router is in odd column: eliminate E to S and N to W



Adaptive Routing – Turn Model

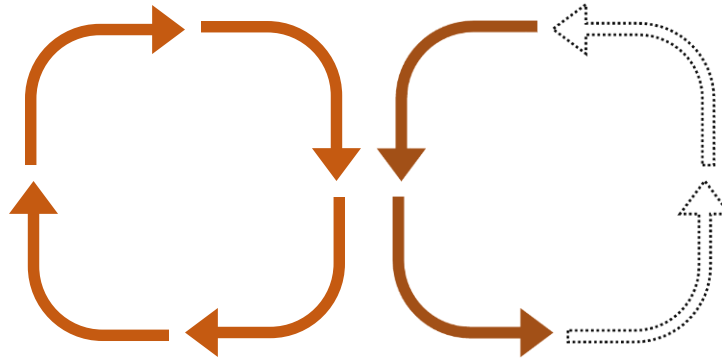
Odd-Even:

- *How about these turn restrictions?*
- If current router is in even column: eliminate N to W and E to N
- If current router is in odd column: eliminate E to S and S to W

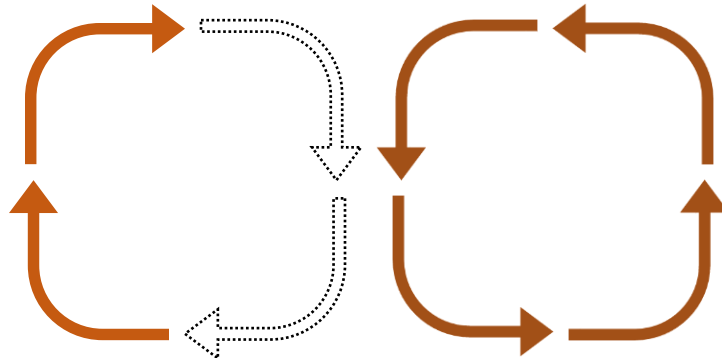
Adaptive Routing – Turn Model

Odd-Even:

- *How about these turn restrictions?*
- If current router is in even column: eliminate N to W and E to N



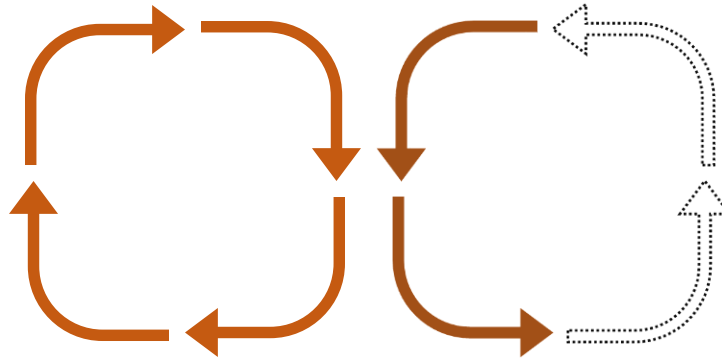
- If current router is in odd column: eliminate E to S and S to W



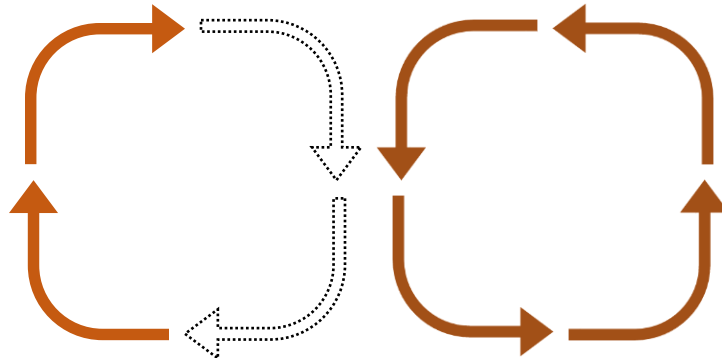
Adaptive Routing – Turn Model

Odd-Even:

- *How about these turn restrictions?* **Not deadlock-free**
- If current router is in even column: eliminate N to W and E to N



- If current router is in odd column: eliminate E to S and S to W



Routing Implementation – Source Tables

Entire route specified at source

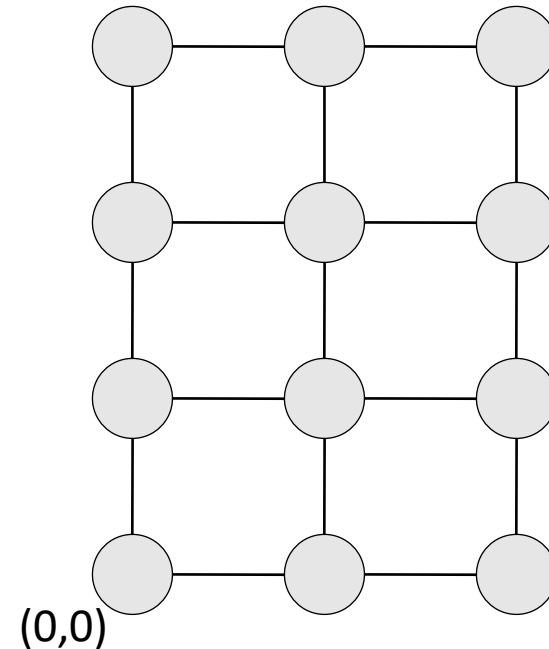
- Avoids per-hop routing latency
- Unable to adjust dynamically to network conditions (i.e., not for adaptive routing)
- Can specify multiple routes per destination for fault tolerance and load balance
- Can support reconfiguration; not fixed to the topology
- Arbitrary-length paths incur storage overhead and packet overhead

Routing Implementation – Source Tables

Entire route specified at source

- Avoids per-hop routing latency
- Unable to adjust dynamically to network conditions (i.e., not for adaptive routing)
- Can specify multiple routes per destination for fault tolerance and load balance
- Can support reconfiguration; not fixed to the topology
- Arbitrary-length paths incur storage overhead and packet overhead

Destination	Route 1	Route 2
00	X	X
10	EX	EX
20	EEX	EEX
01	NX	NX
11	NEX	ENX
21	NEEX	ENEX
02	NNX	NNX
12	ENNX	NNEX
22	EENNX	NNEEX
03	NNNX	NNNX
13	NENNX	ENNNX
23	EENNNX	NNNEEX



Routing Implementation – Node Tables

Store only next direction at each router

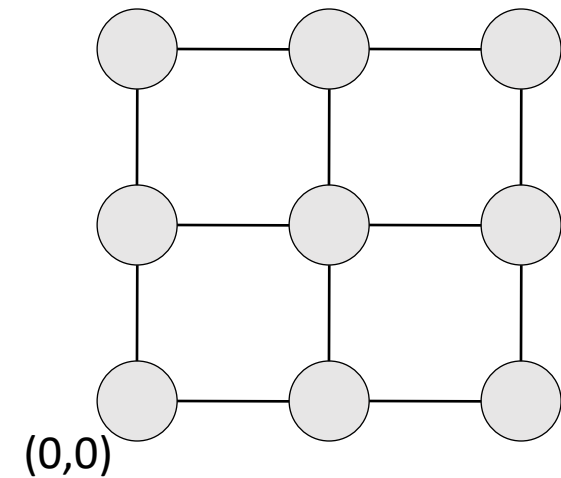
- Adds per-hop routing latency
- Can adapt to network conditions
- Can specify multiple possible output ports or select randomly

Routing Implementation – Node Tables

Store only next direction at each router

- Adds per-hop routing latency
- Can adapt to network conditions
- Can specify multiple possible output ports or select randomly

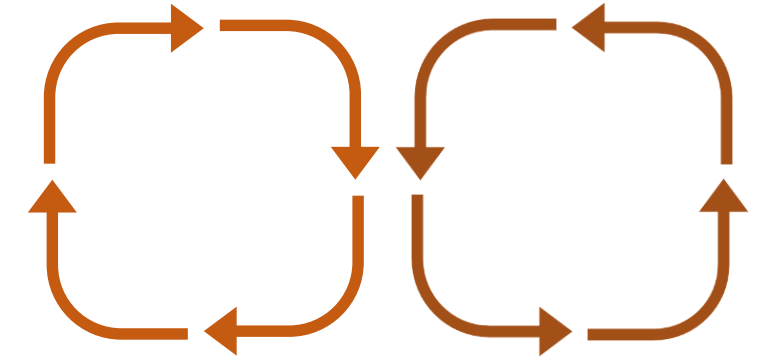
To									
From	00	01	02	10	11	12	20	21	22
00	X -	N -	N -	E -	E N	E N	E -	E N	E N
01	S -	X -	N -	E S	E -	E N	E S	E -	E N
02	S -	S -	X -	E S	E S	E -	E S	E S	E -
10	W -	W -	W -	X -	N -	N -	E -	E N	E N
11	W -	W -	W -	S -	X -	N -	E S	E -	E N
12	W -	W -	W -	S -	S -	X -	E S	E S	E -
20	W -	W -	W -	W -	W -	W -	X -	N -	N -
21	W -	W -	W -	W -	W -	W -	S -	X -	N -
22	W -	W -	W -	W -	W -	W -	S -	S -	X -



Routing Implementation – Node Tables

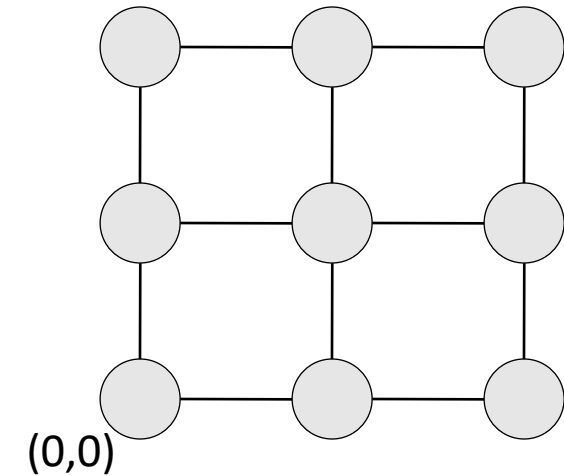
Store only next direction at each router

- Adds per-hop routing latency
- Can adapt to network conditions
- Can specify multiple possible output ports or select randomly



which turns are disallowed?

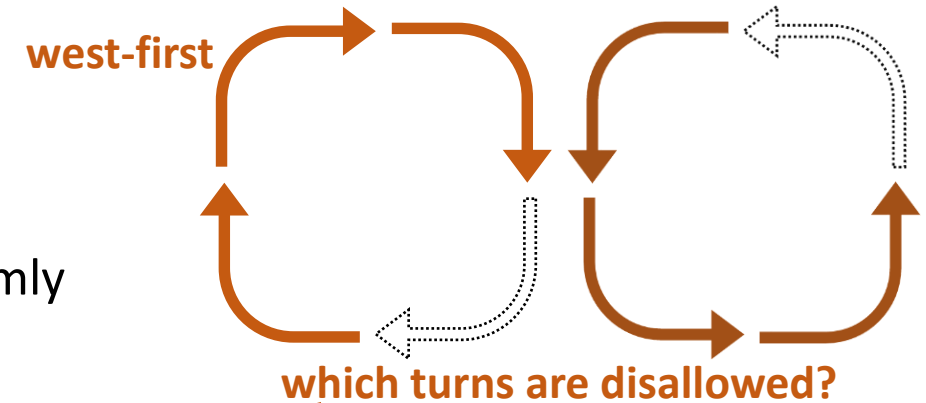
To									
From	00	01	02	10	11	12	20	21	22
00	X -	N -	N -	E -	E N	E N	E -	E N	E N
01	S -	X -	N -	E S	E -	E N	E S	E -	E N
02	S -	S -	X -	E S	E S	E -	E S	E S	E -
10	W -	W -	W -	X -	N -	N -	E -	E N	E N
11	W -	W -	W -	S -	X -	N -	E S	E -	E N
12	W -	W -	W -	S -	S -	X -	E S	E S	E -
20	W -	W -	W -	W -	W -	W -	X -	N -	N -
21	W -	W -	W -	W -	W -	W -	S -	X -	N -
22	W -	W -	W -	W -	W -	W -	S -	S -	X -



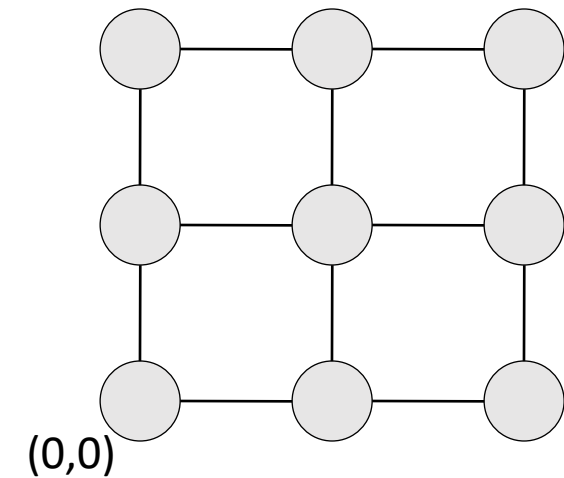
Routing Implementation – Node Tables

Store only next direction at each router

- Adds per-hop routing latency
- Can adapt to network conditions
- Can specify multiple possible output ports or select randomly



To									
From	00	01	02	10	11	12	20	21	22
00	X -	N -	N -	E -	E N	E N	E -	E N	E N
01	S -	X -	N -	E S	E -	E N	E S	E -	E N
02	S -	S -	X -	E S	E S	E -	E S	E S	E -
10	W -	W -	W -	X -	N -	N -	E -	E N	E N
11	W -	W -	W -	S -	X -	N -	E S	E -	E N
12	W -	W -	W -	S -	S -	X -	E S	E S	E -
20	W -	W -	W -	W -	W -	W -	X -	N -	N -
21	W -	W -	W -	W -	W -	W -	S -	X -	N -
22	W -	W -	W -	W -	W -	W -	S -	S -	X -



Routing Implementation – Combinational

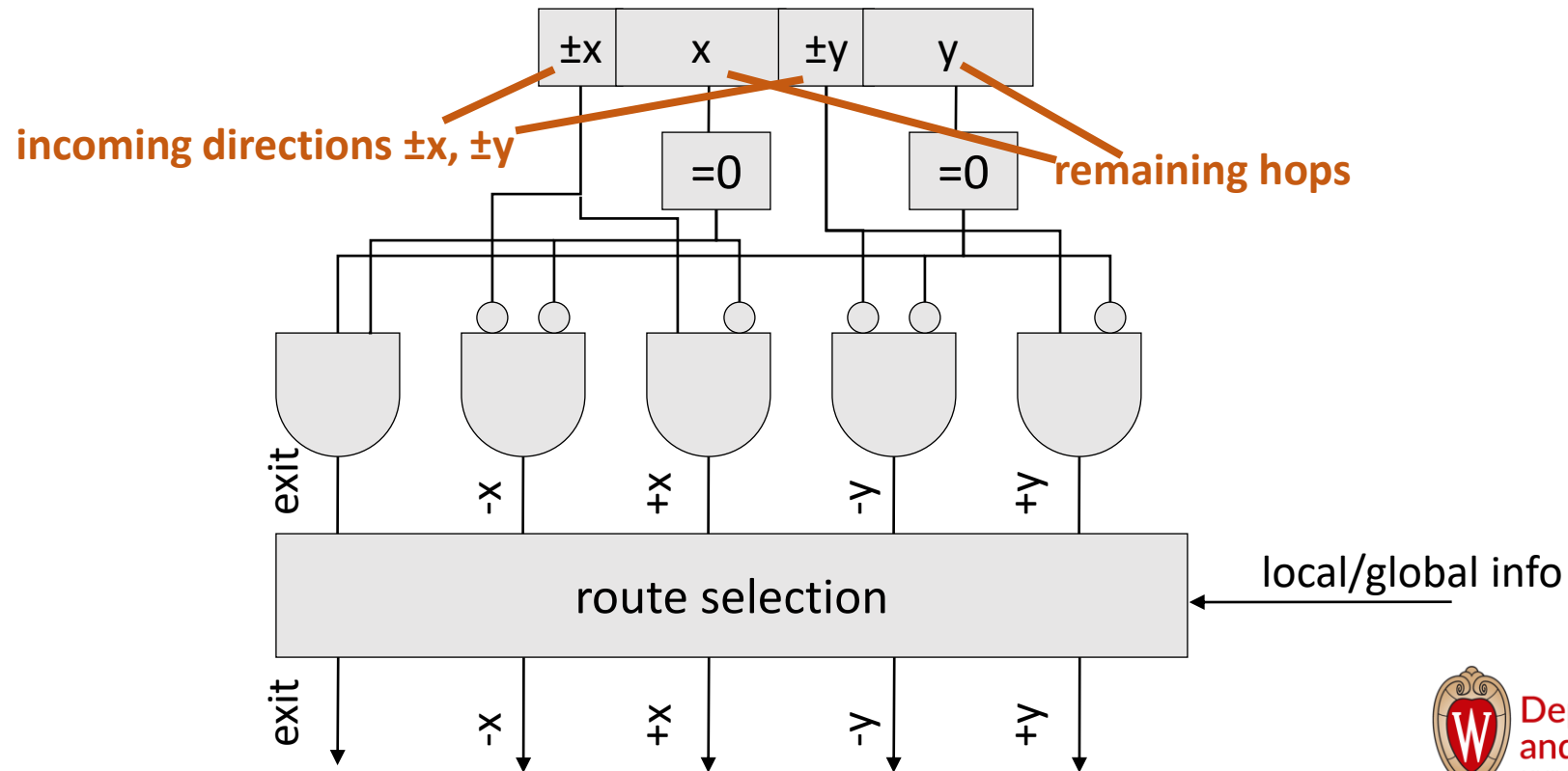
Compute routes using combinational circuits

- Simple to implement; low router overhead
- Fixed to topology and routing algorithm; limits fault tolerance

Routing Implementation – Combinational

Compute routes using combinational circuits

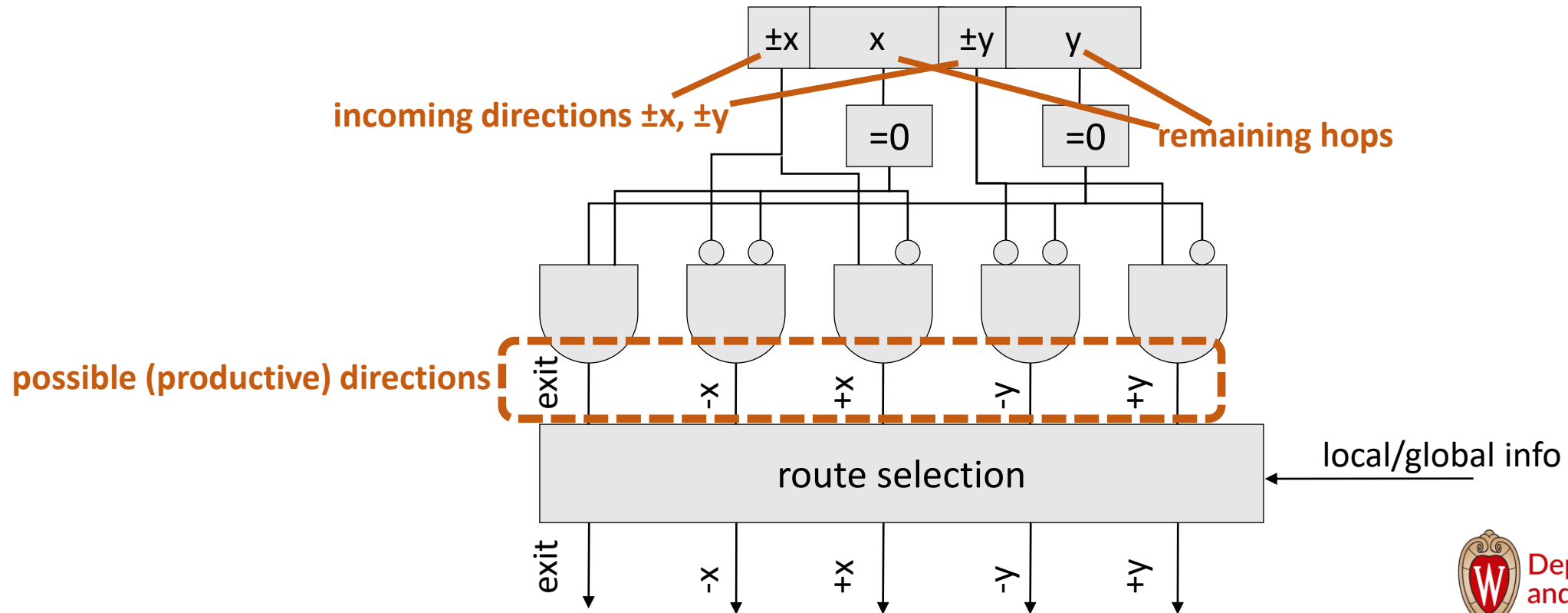
- Simple to implement; low router overhead
- Fixed to topology and routing algorithm; limits fault tolerance



Routing Implementation – Combinational

Compute routes using combinational circuits

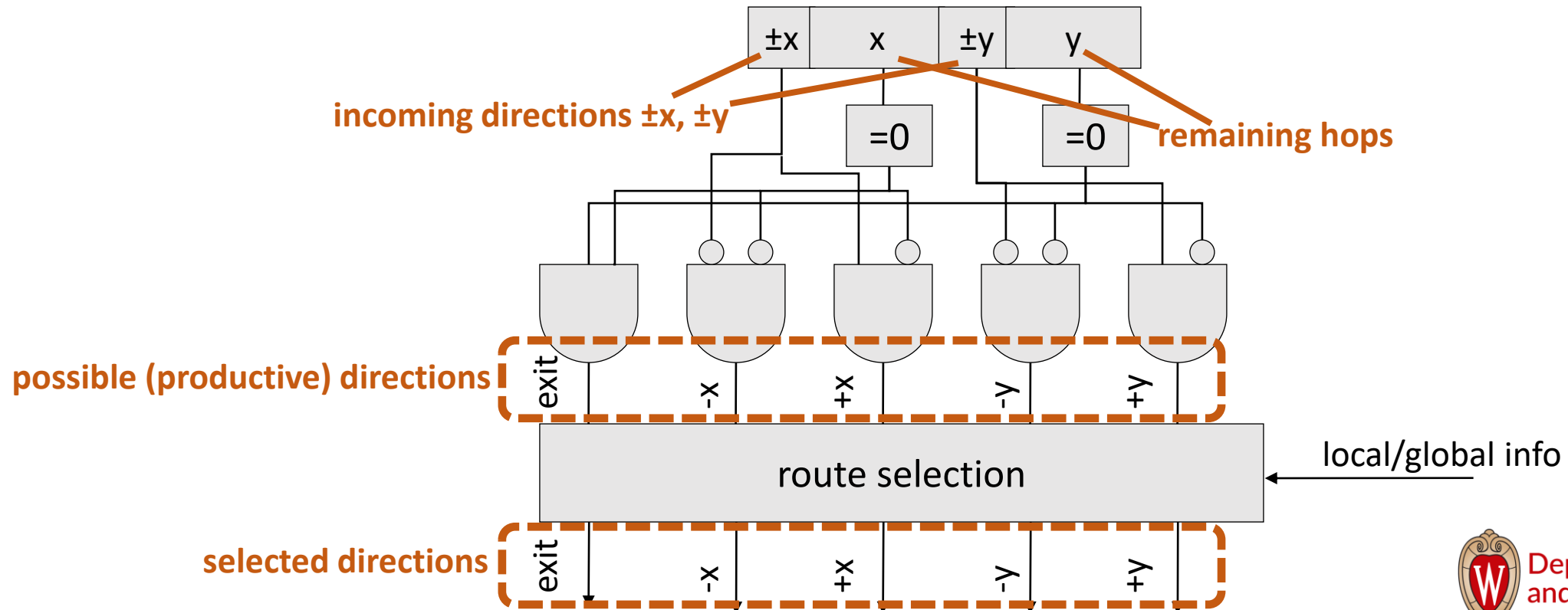
- Simple to implement; low router overhead
- Fixed to topology and routing algorithm; limits fault tolerance



Routing Implementation – Combinational

Compute routes using combinational circuits

- Simple to implement; low router overhead
- Fixed to topology and routing algorithm; limits fault tolerance



Network-On-Chip

Key design decisions:

Topology

- Where to place routers/nodes and how to link them together

Routing

- Which set of links to traverse between source-destination pairs

Flow Control

- What granularity to move messages through links and routers

Router Microarchitecture

- How to pipeline and arbitrate incoming messages in routers



Flow Control

Flow control determines allocation of resources to messages as they traverse the network

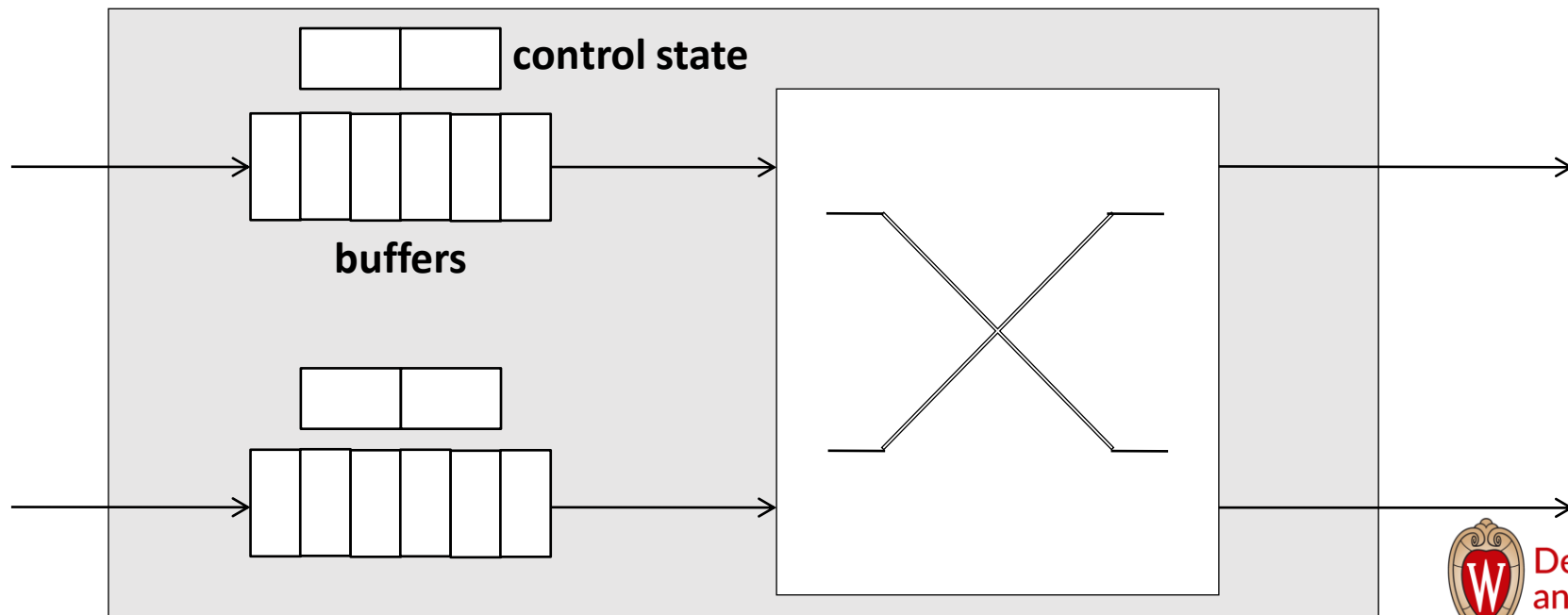
- Analogous to traffic lights
- Allocation of buffers and links
- Goal is to propagate traffic fairly
- Significant impact on throughput and latency

Flow Control

Buffers hold packets (or flits) waiting for available link bandwidth

Control state records:

- Allocation of links and buffers to packets
- Current state of packet traversing router



Flow Control

Messages: composed of one or more packets

- In shared-memory CMPs, usually one packet per coherence message

Packets: composed of one or more flits

- Messages are routed at packet granularity

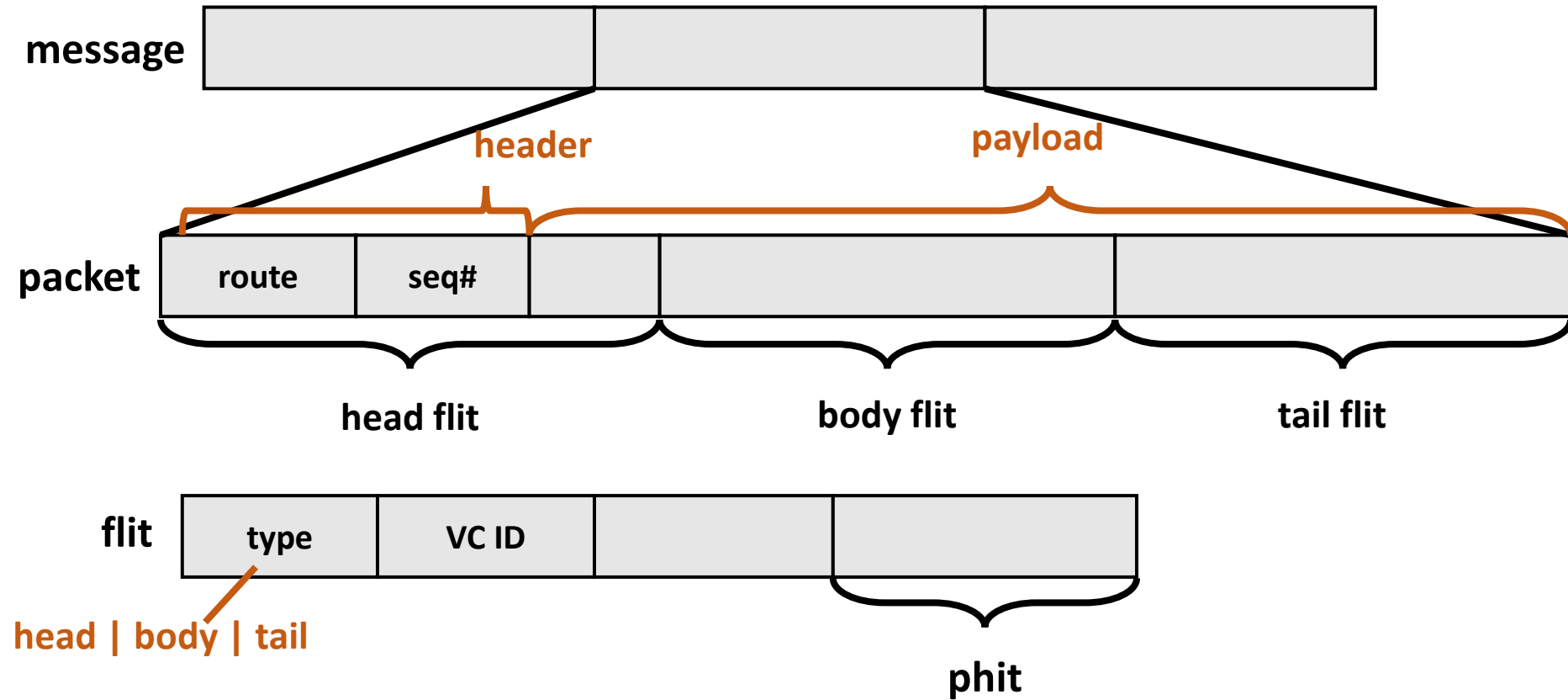
Flit: flow control unit

- Resources can be allocated at flit granularity

Phit: physical unit

- Subdivide flit into chunks equal to the link width
- On-chip wiring is typically abundant: phit size = flit size

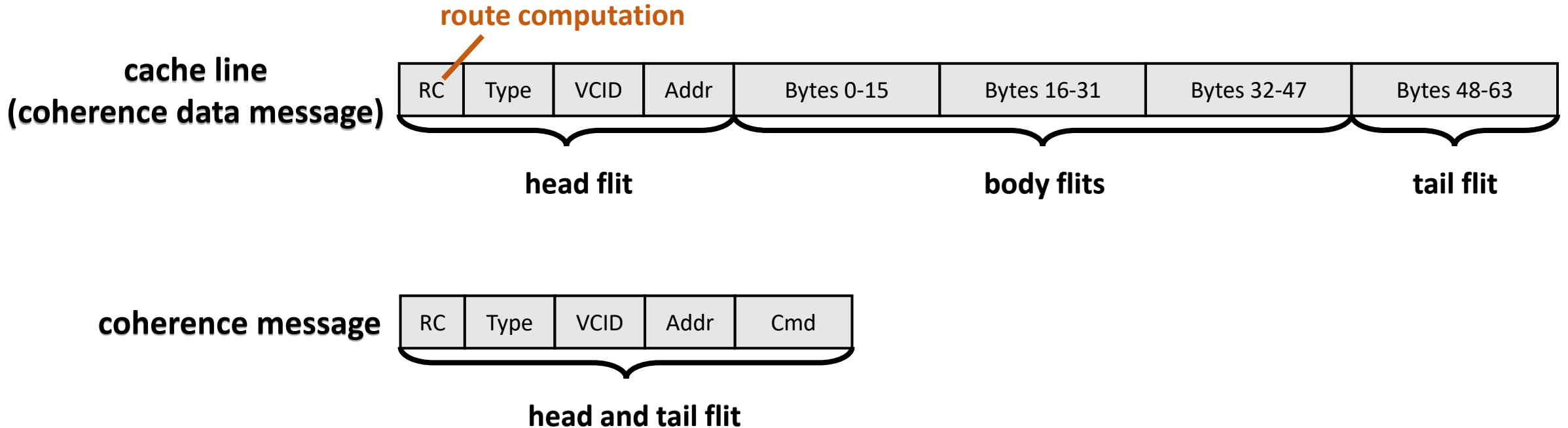
Flow Control



Flow Control

Packets contain route information (i.e., destination)

- Flits may not; all flits of a packet must take the same route



Flow Control – Switching

Message-based:

Allocation made at message granularity

- e.g., circuit-switching

Packet-based:

Allocation made at packet granularity

- e.g., store and forward, virtual cut through

Flit-based:

Allocation made at flit granularity

- e.g., wormhole

Message-Based Flow Control

Coarsest granularity of flow control

Circuit-Switching:

- Pre-allocates resources across multiple hops
- Probe sent into network to reserve resources
- Along precomputed path from source to destination
- Links are allocated; buffers are not necessary

Circuit-Switching

Message performs routing and allocation at source and sends out probe to reserve path (circuit) to destination

- No need to perform routing nor allocation at each hop
- Good for transferring large amounts of data
- Trades off high circuit setup cost for low per-hop overheads

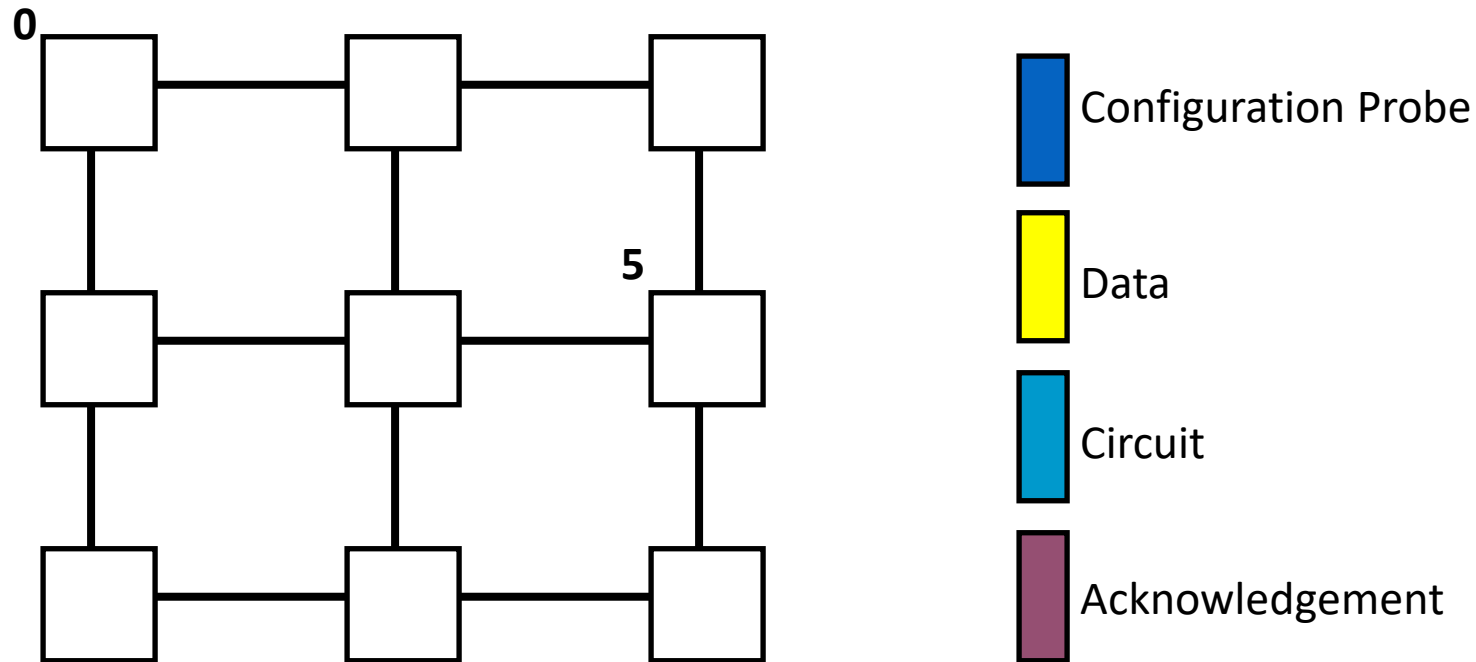
No other message can use allocated resources until transfer is complete

- Network throughput can suffer due to setup and hold time of reservations
- Link utilization can be low; links along path are idle until setup is complete

Circuit-Switching

e.g., Message from 0 to 5

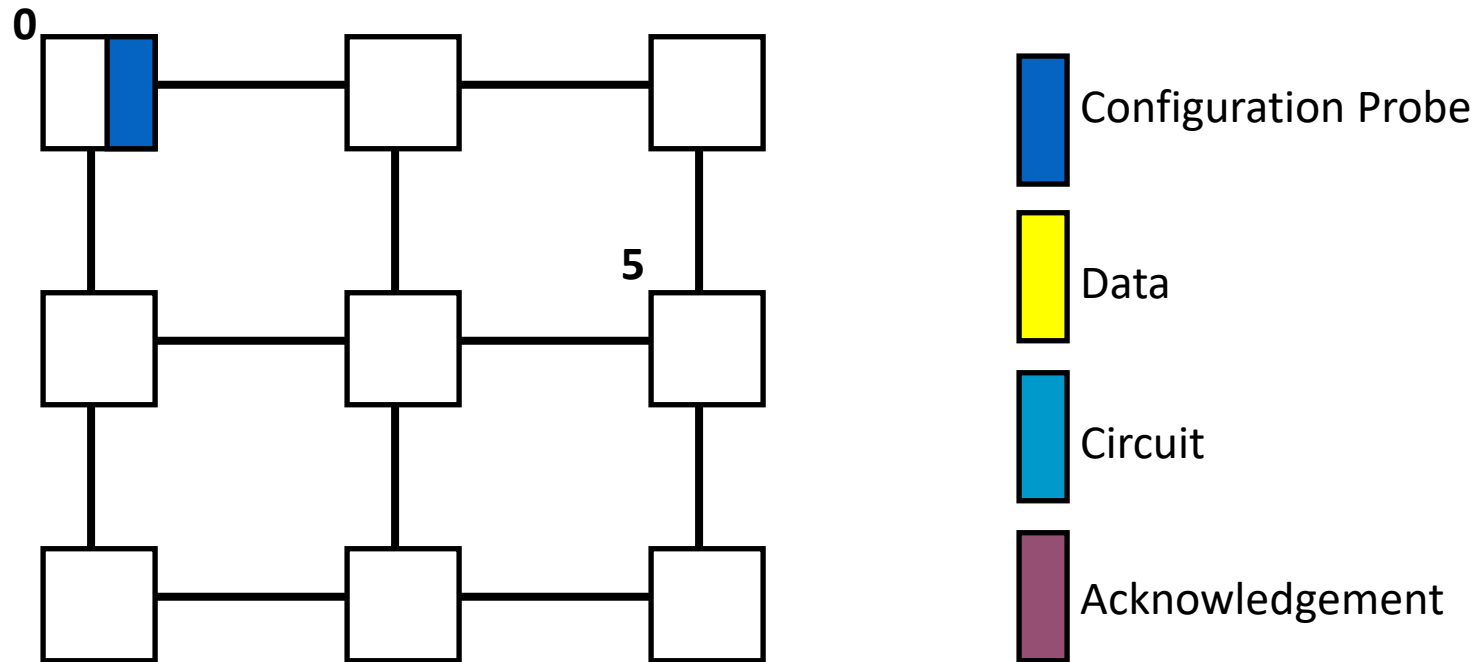
- Significant latency overhead prior to data transfer
- Data transfer does not pay per-hop overhead for routing and allocation



Circuit-Switching

e.g., Message from 0 to 5

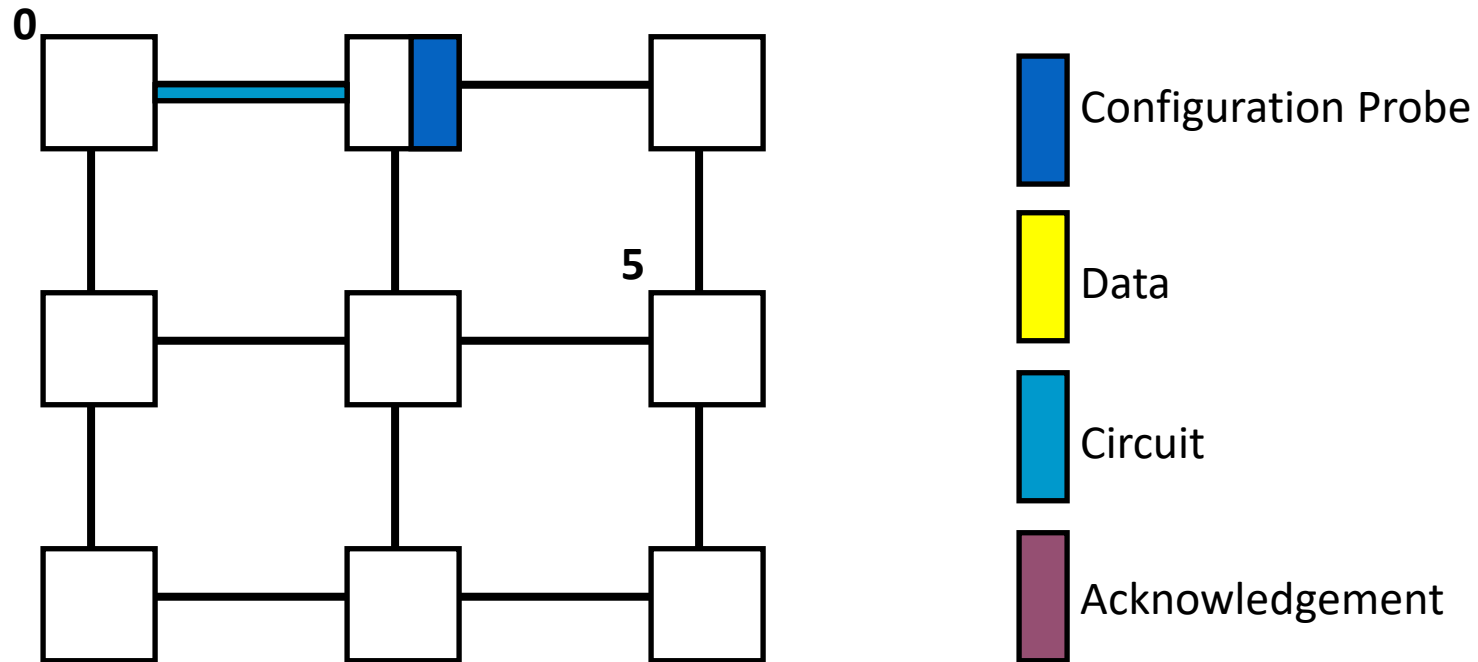
- Significant latency overhead prior to data transfer
- Data transfer does not pay per-hop overhead for routing and allocation



Circuit-Switching

e.g., Message from 0 to 5

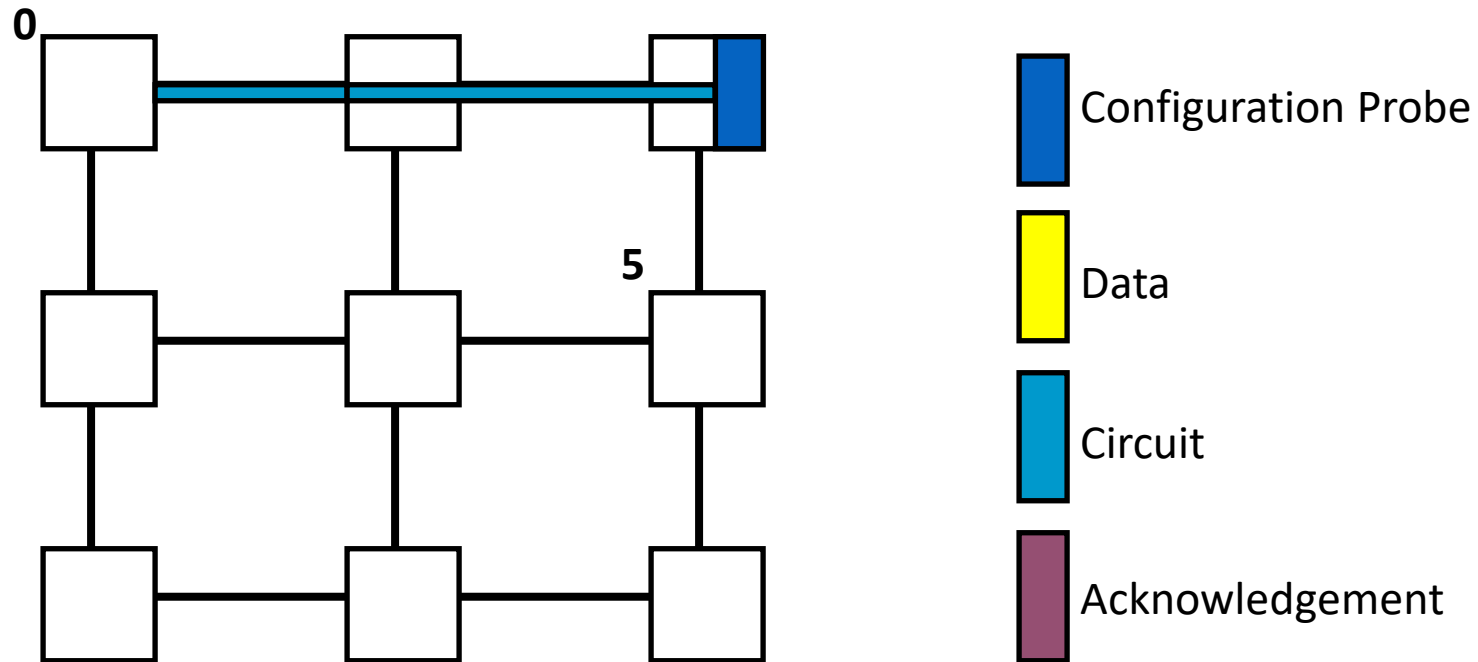
- Significant latency overhead prior to data transfer
- Data transfer does not pay per-hop overhead for routing and allocation



Circuit-Switching

e.g., Message from 0 to 5

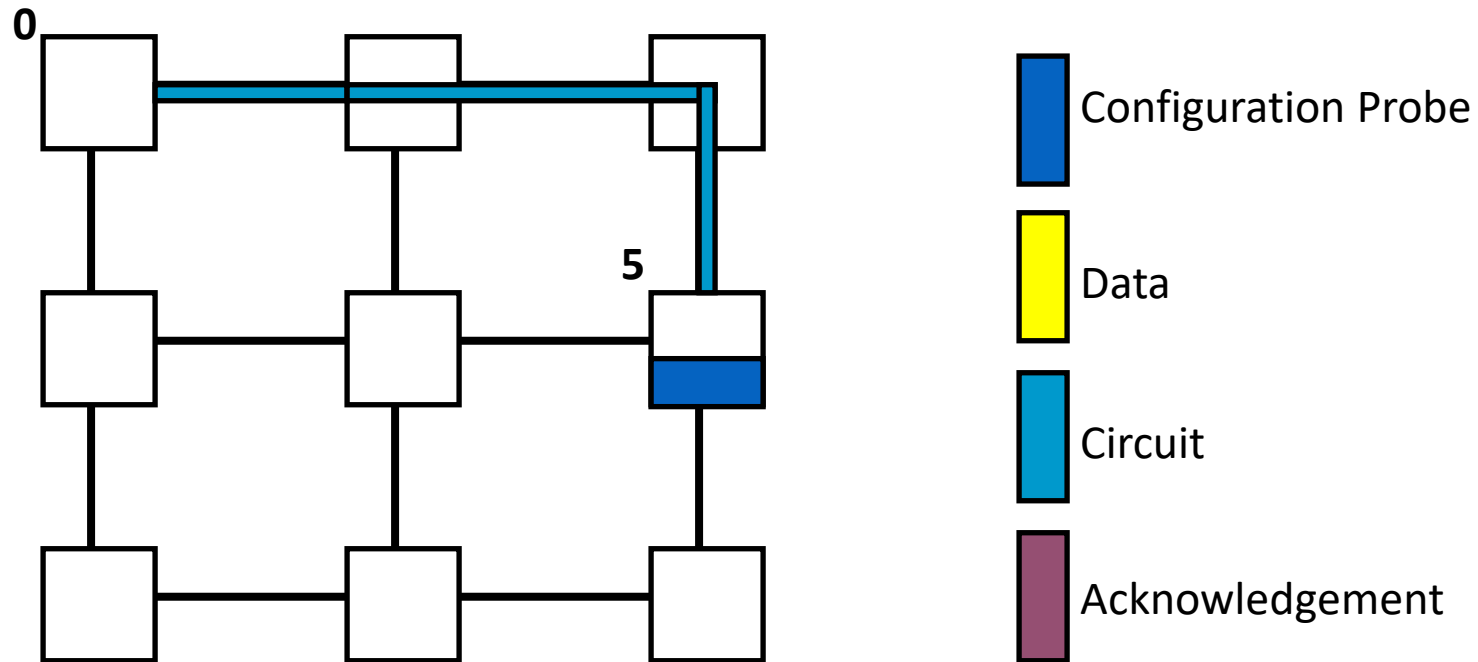
- Significant latency overhead prior to data transfer
- Data transfer does not pay per-hop overhead for routing and allocation



Circuit-Switching

e.g., Message from 0 to 5

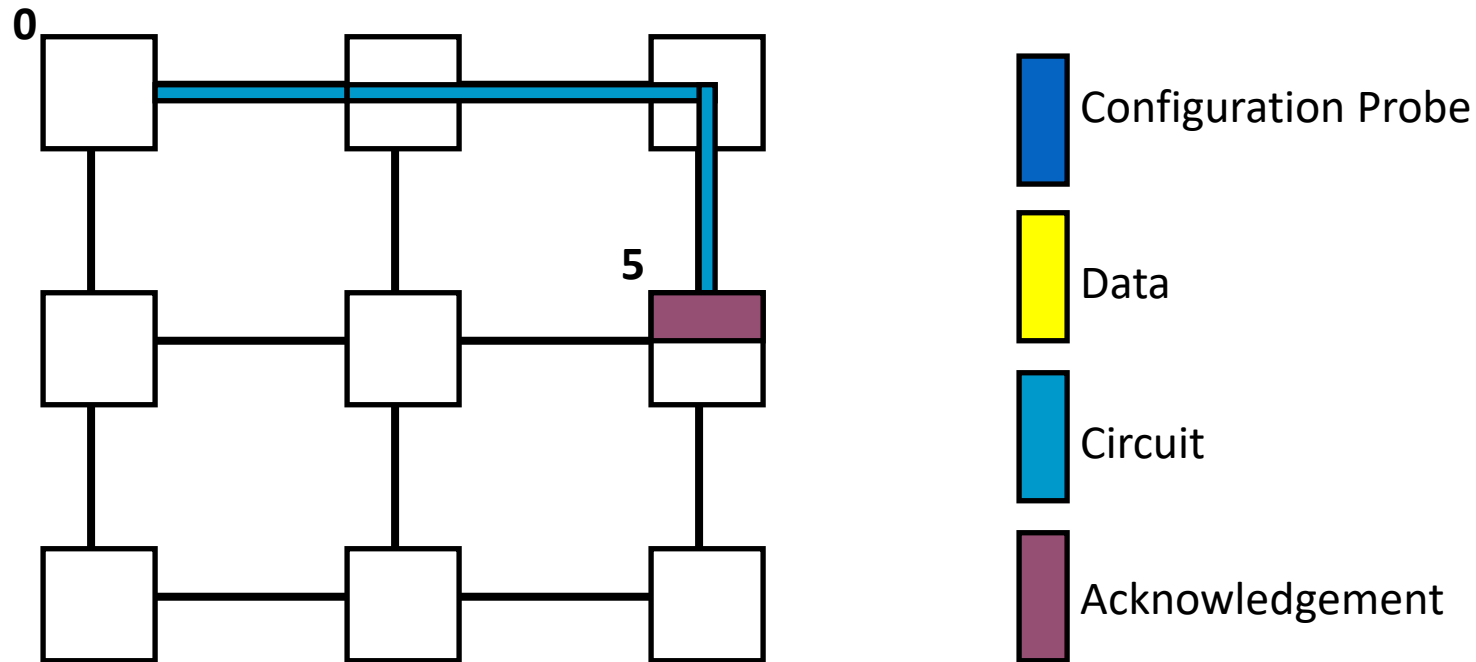
- Significant latency overhead prior to data transfer
- Data transfer does not pay per-hop overhead for routing and allocation



Circuit-Switching

e.g., Message from 0 to 5

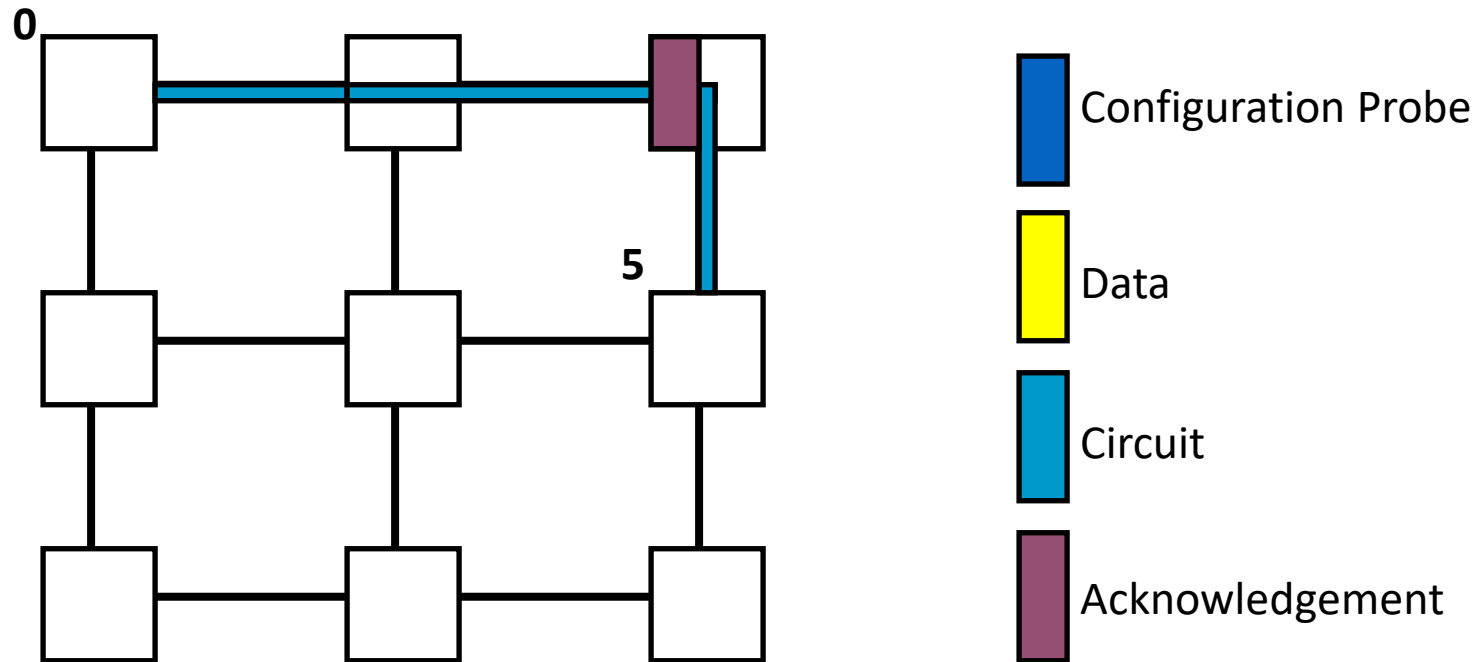
- Significant latency overhead prior to data transfer
- Data transfer does not pay per-hop overhead for routing and allocation



Circuit-Switching

e.g., Message from 0 to 5

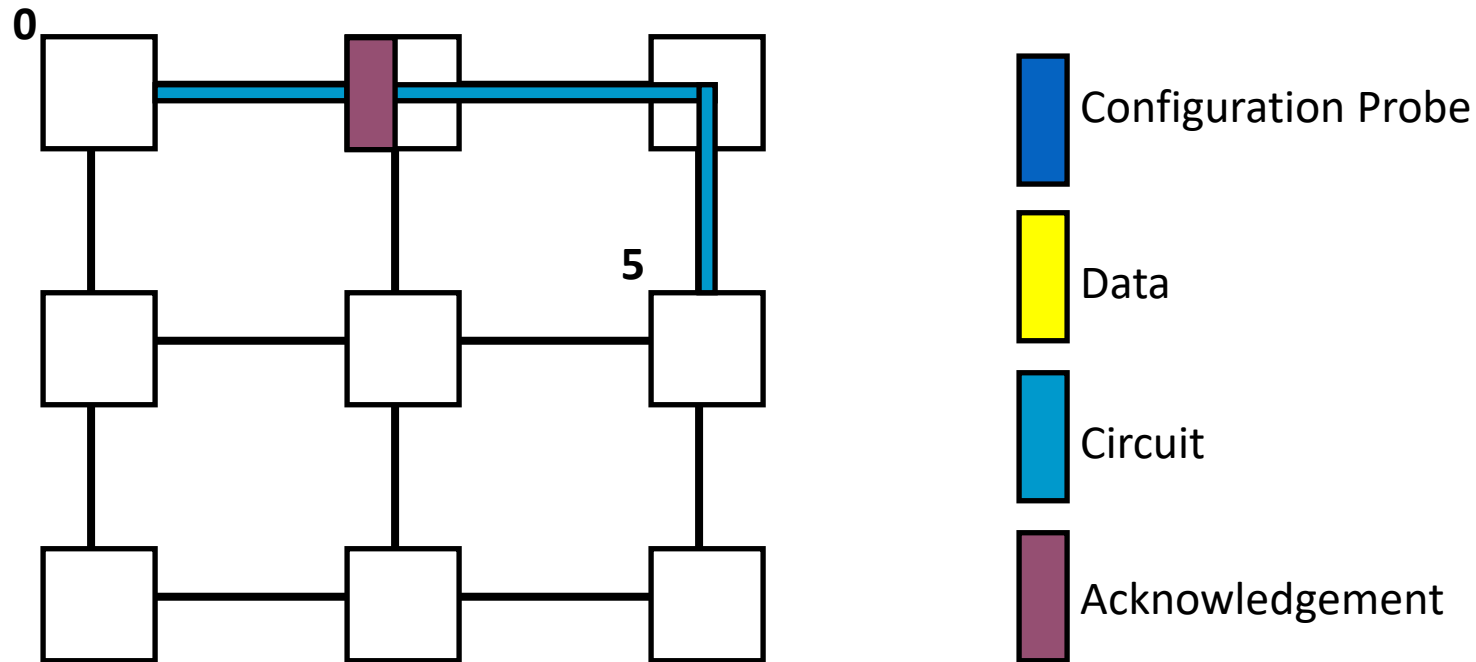
- Significant latency overhead prior to data transfer
- Data transfer does not pay per-hop overhead for routing and allocation



Circuit-Switching

e.g., Message from 0 to 5

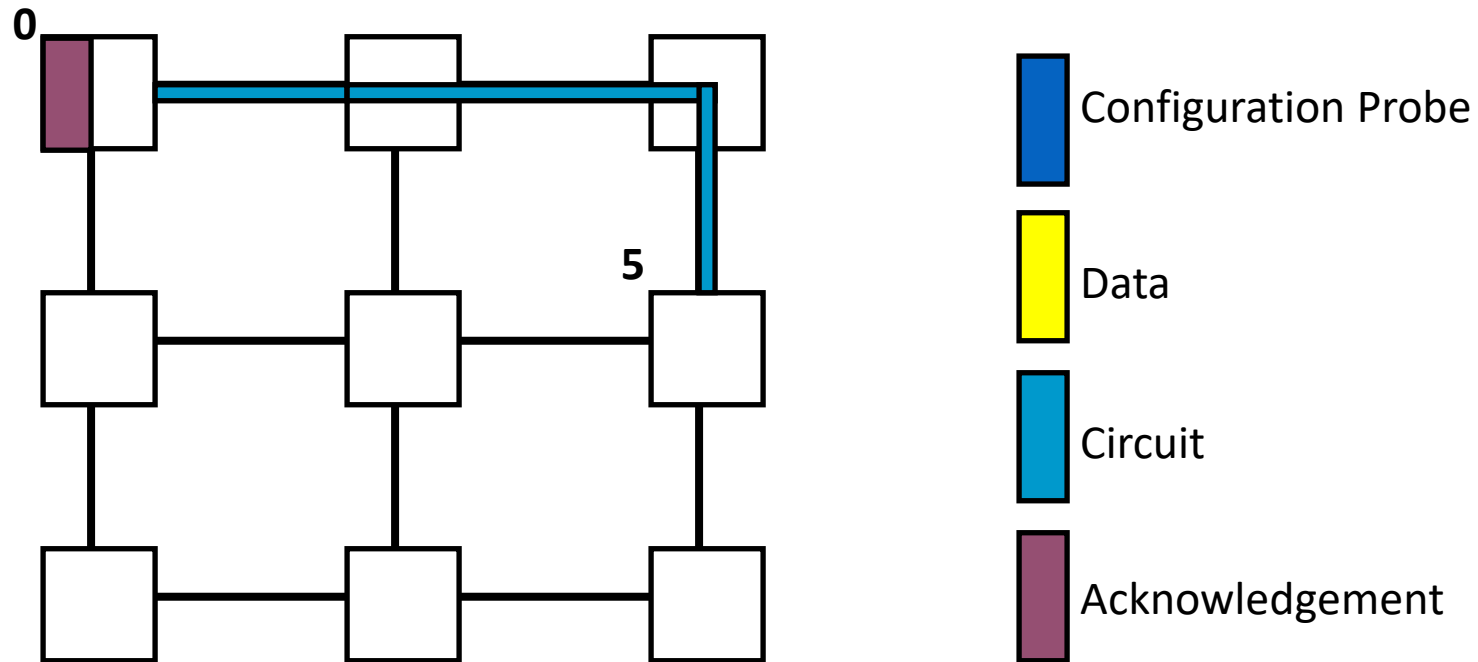
- Significant latency overhead prior to data transfer
- Data transfer does not pay per-hop overhead for routing and allocation



Circuit-Switching

e.g., Message from 0 to 5

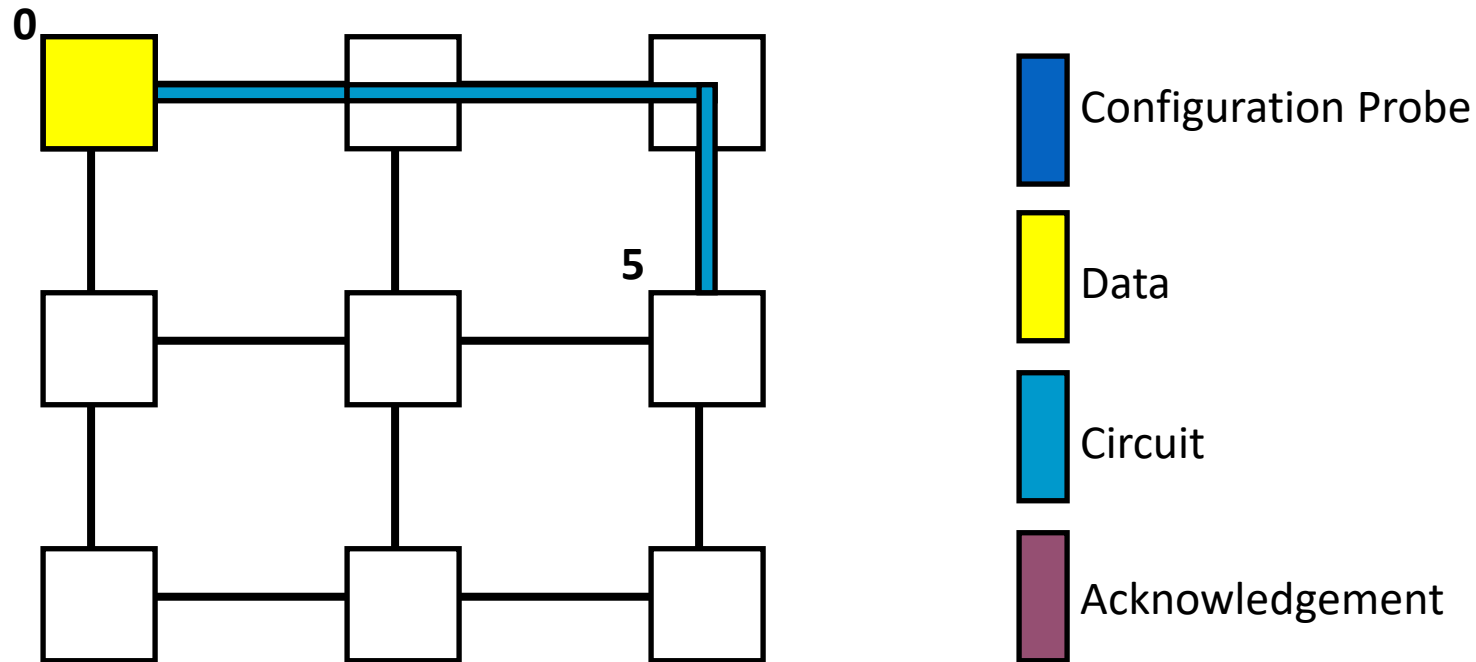
- Significant latency overhead prior to data transfer
- Data transfer does not pay per-hop overhead for routing and allocation



Circuit-Switching

e.g., Message from 0 to 5

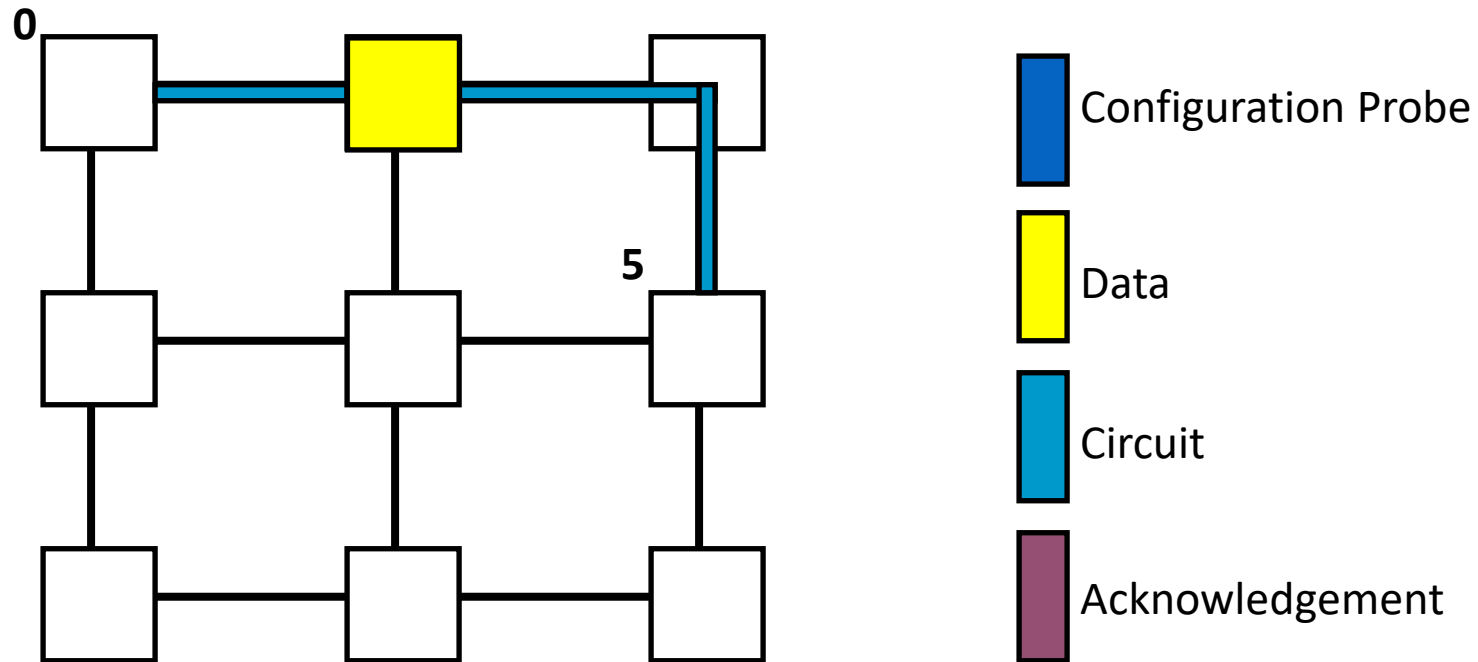
- Significant latency overhead prior to data transfer
- Data transfer does not pay per-hop overhead for routing and allocation



Circuit-Switching

e.g., Message from 0 to 5

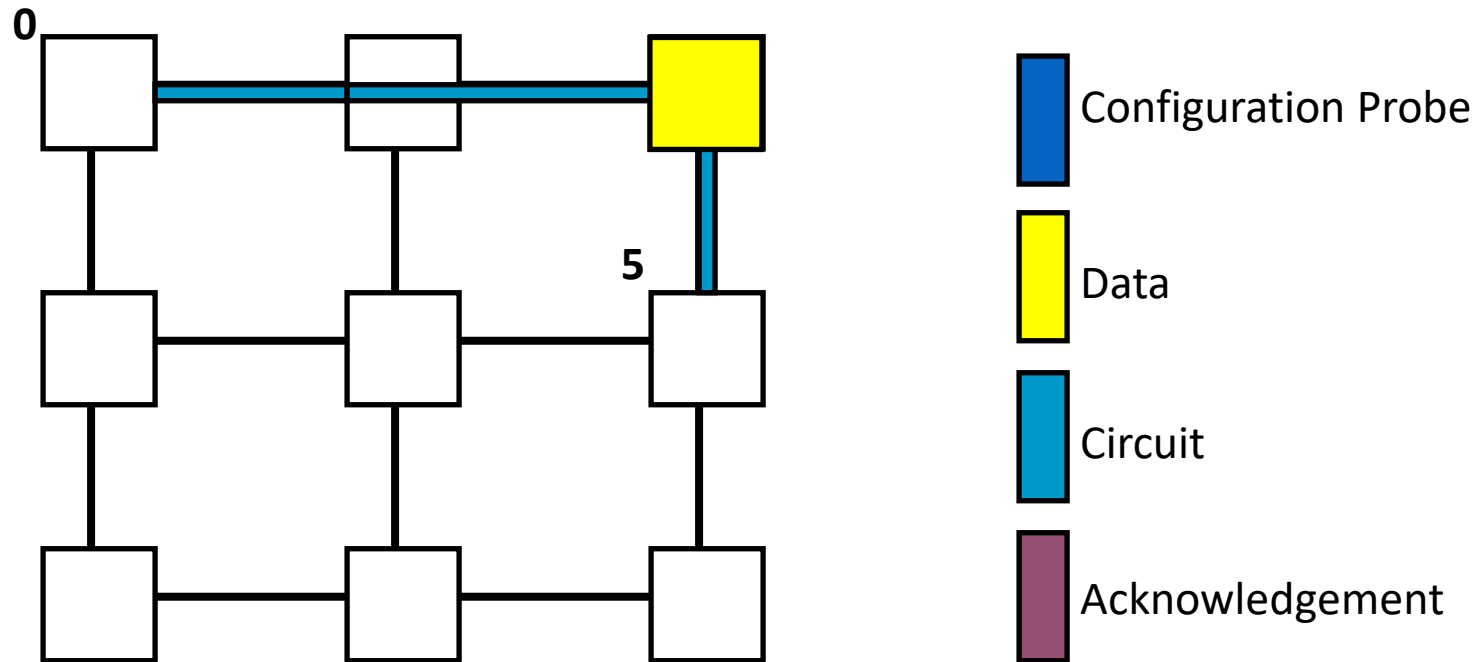
- Significant latency overhead prior to data transfer
- Data transfer does not pay per-hop overhead for routing and allocation



Circuit-Switching

e.g., Message from 0 to 5

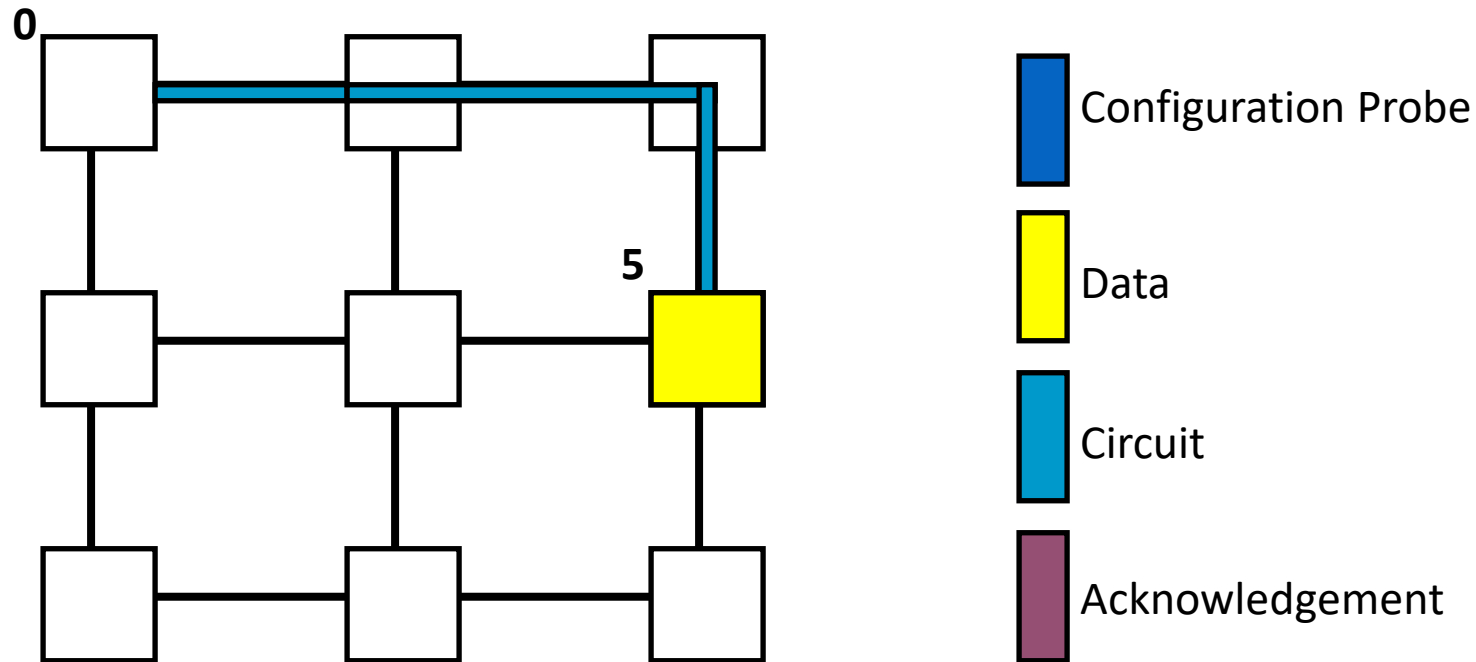
- Significant latency overhead prior to data transfer
- Data transfer does not pay per-hop overhead for routing and allocation



Circuit-Switching

e.g., Message from 0 to 5

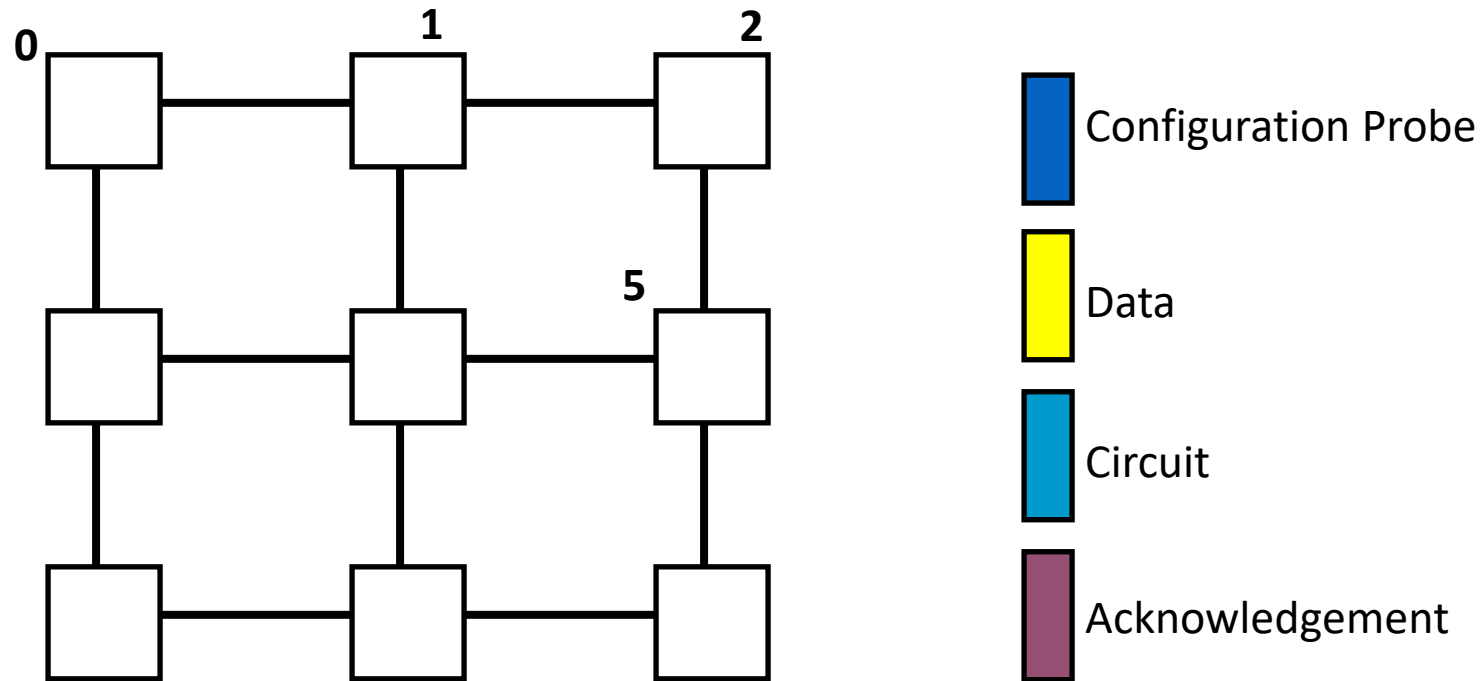
- Significant latency overhead prior to data transfer
- Data transfer does not pay per-hop overhead for routing and allocation



Circuit-Switching

e.g., Message from 0 to 5, and message from 1 to 2

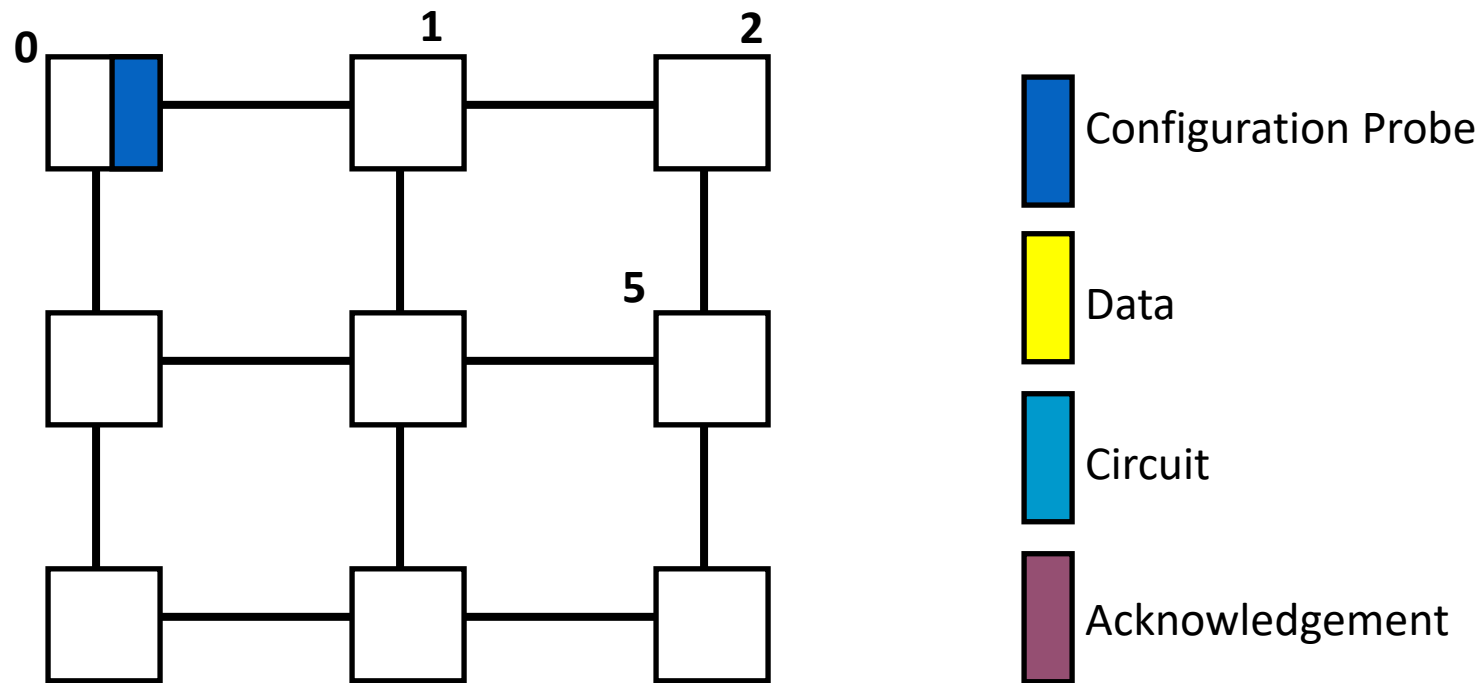
- Long waiting time when there is contention



Circuit-Switching

e.g., Message from 0 to 5, and message from 1 to 2

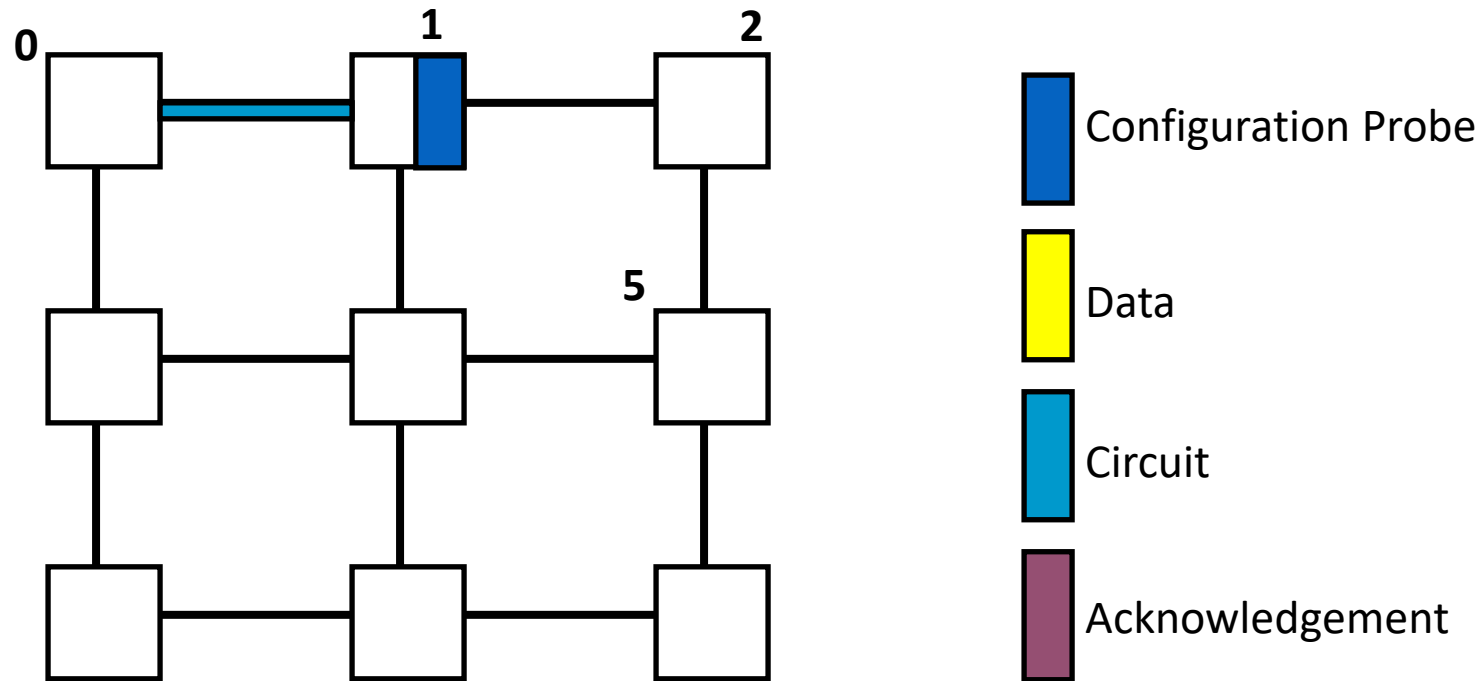
- Long waiting time when there is contention



Circuit-Switching

e.g., Message from 0 to 5, and message from 1 to 2

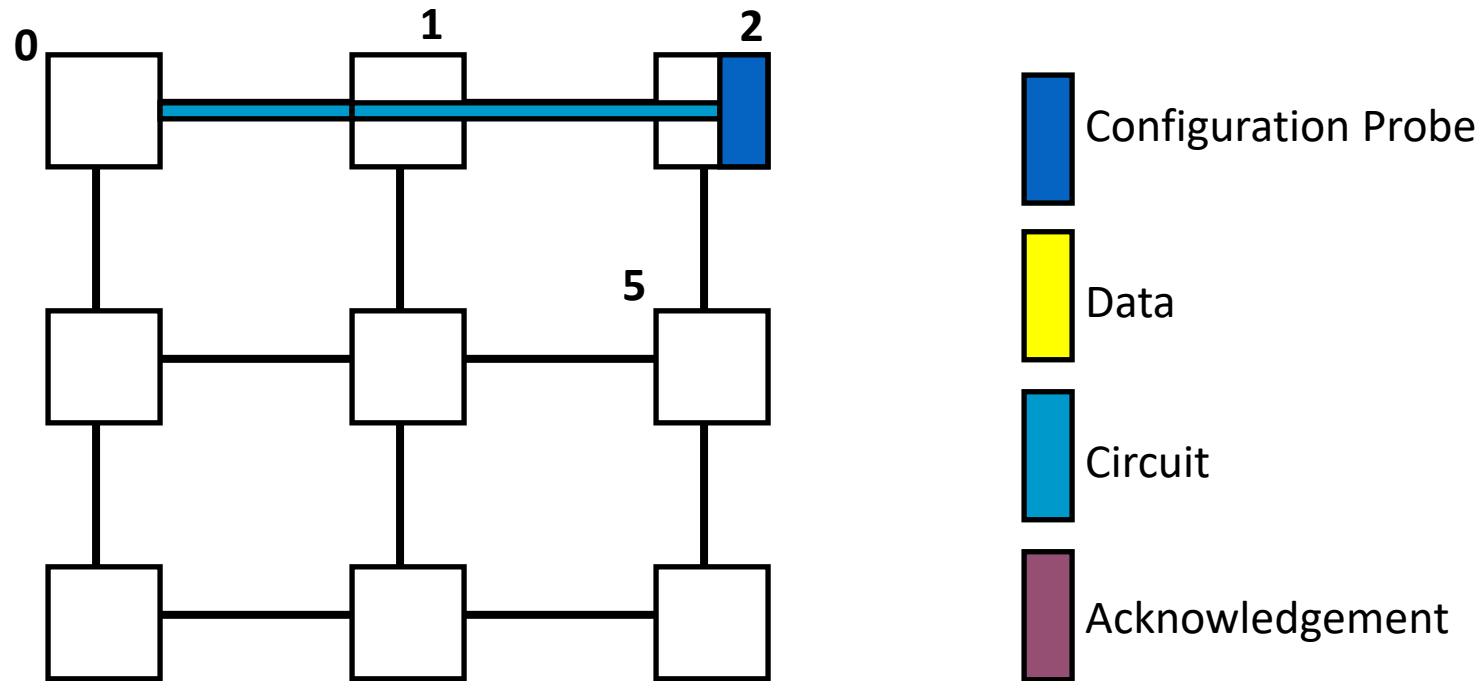
- Long waiting time when there is contention



Circuit-Switching

e.g., Message from 0 to 5, and message from 1 to 2

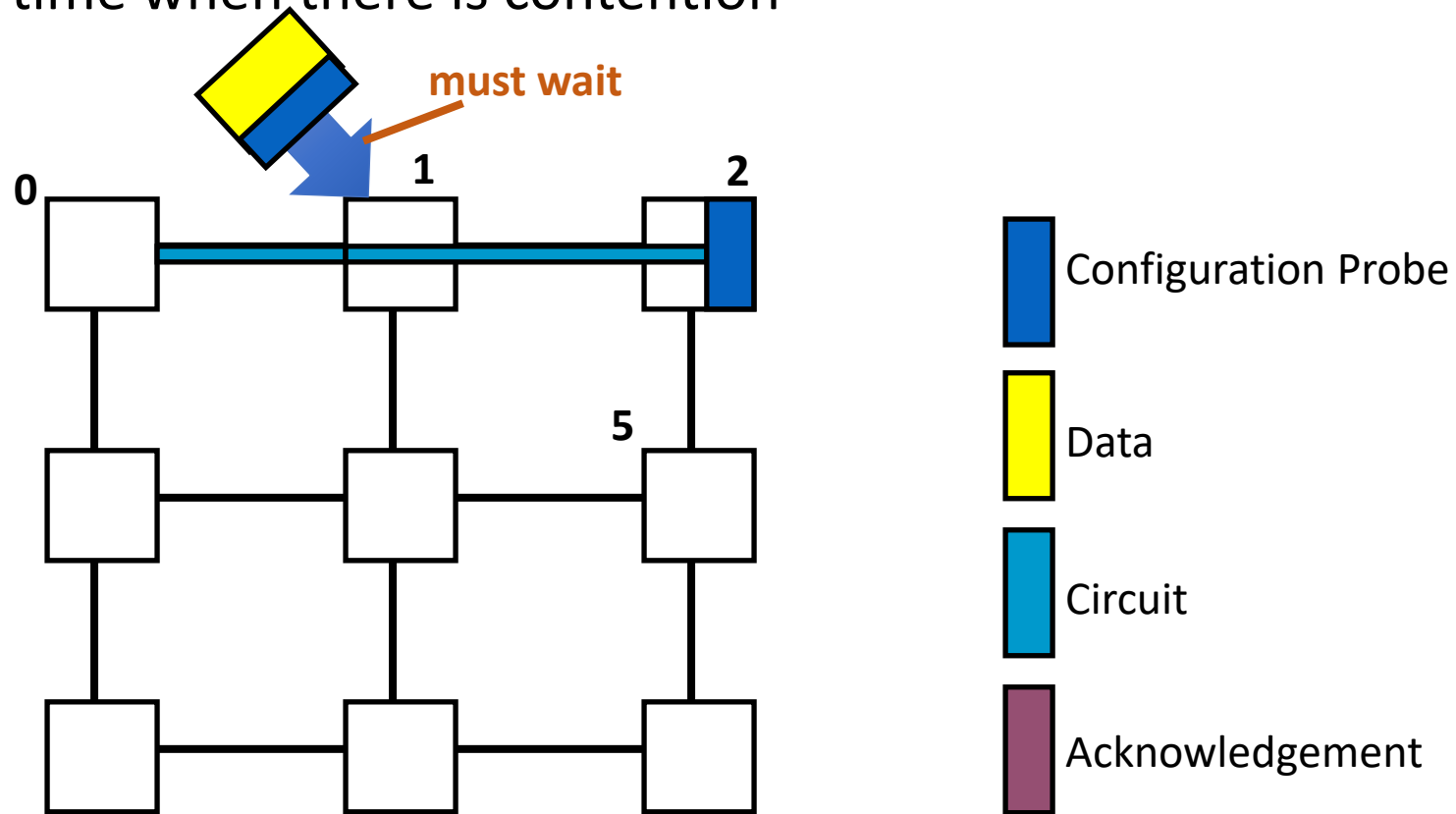
- Long waiting time when there is contention



Circuit-Switching

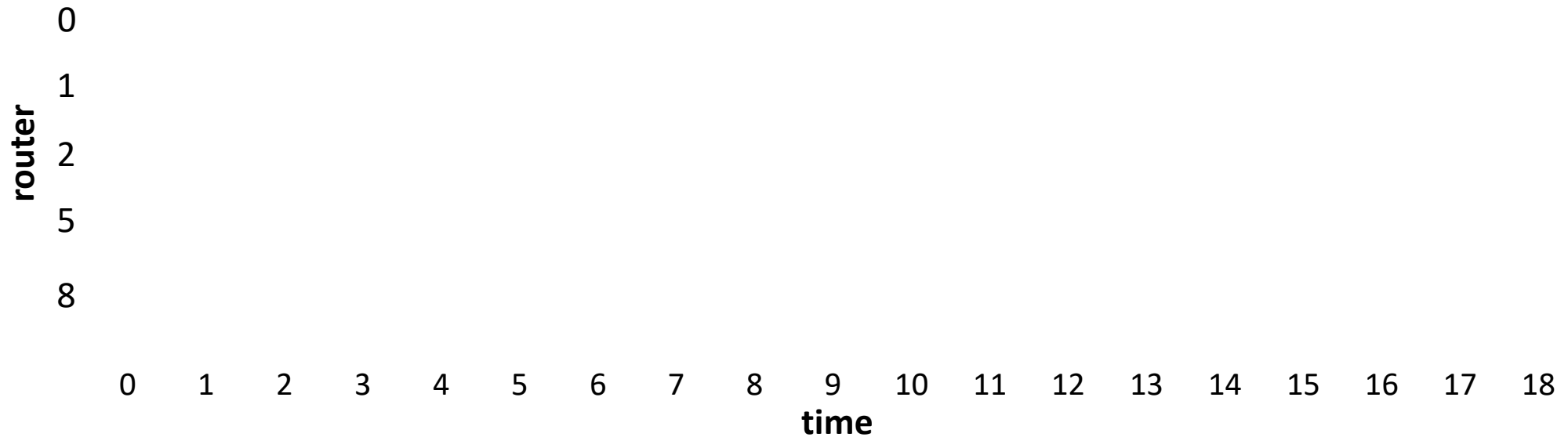
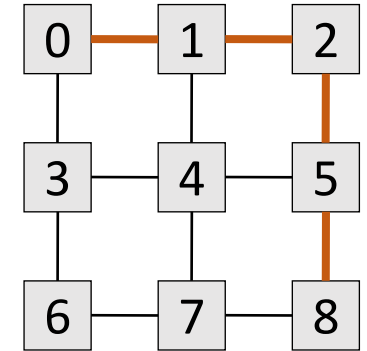
e.g., Message from 0 to 5, and message from 1 to 2

- Long waiting time when there is contention



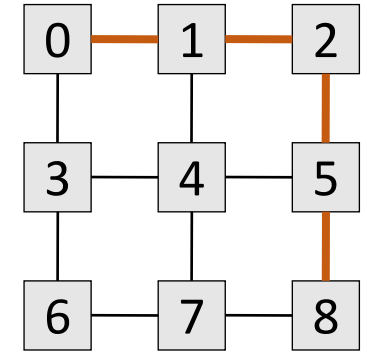
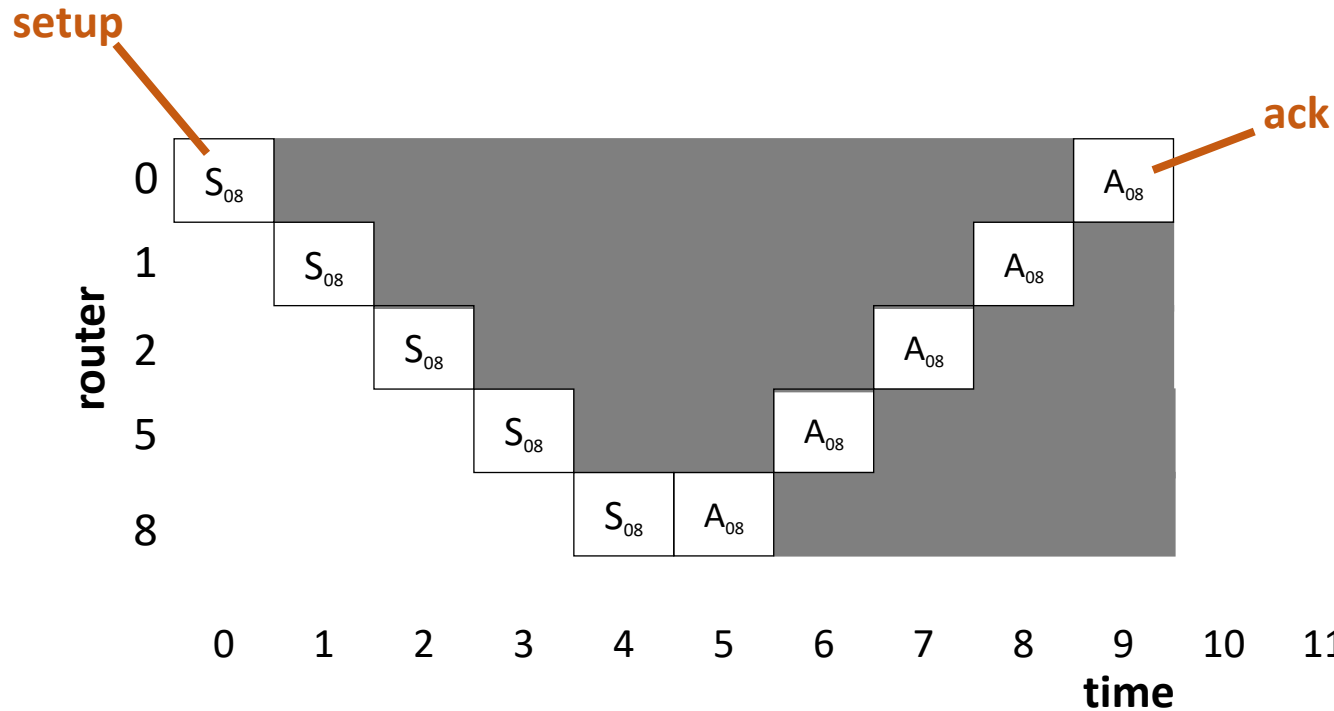
Circuit-Switching

e.g., Message from 0 to 8



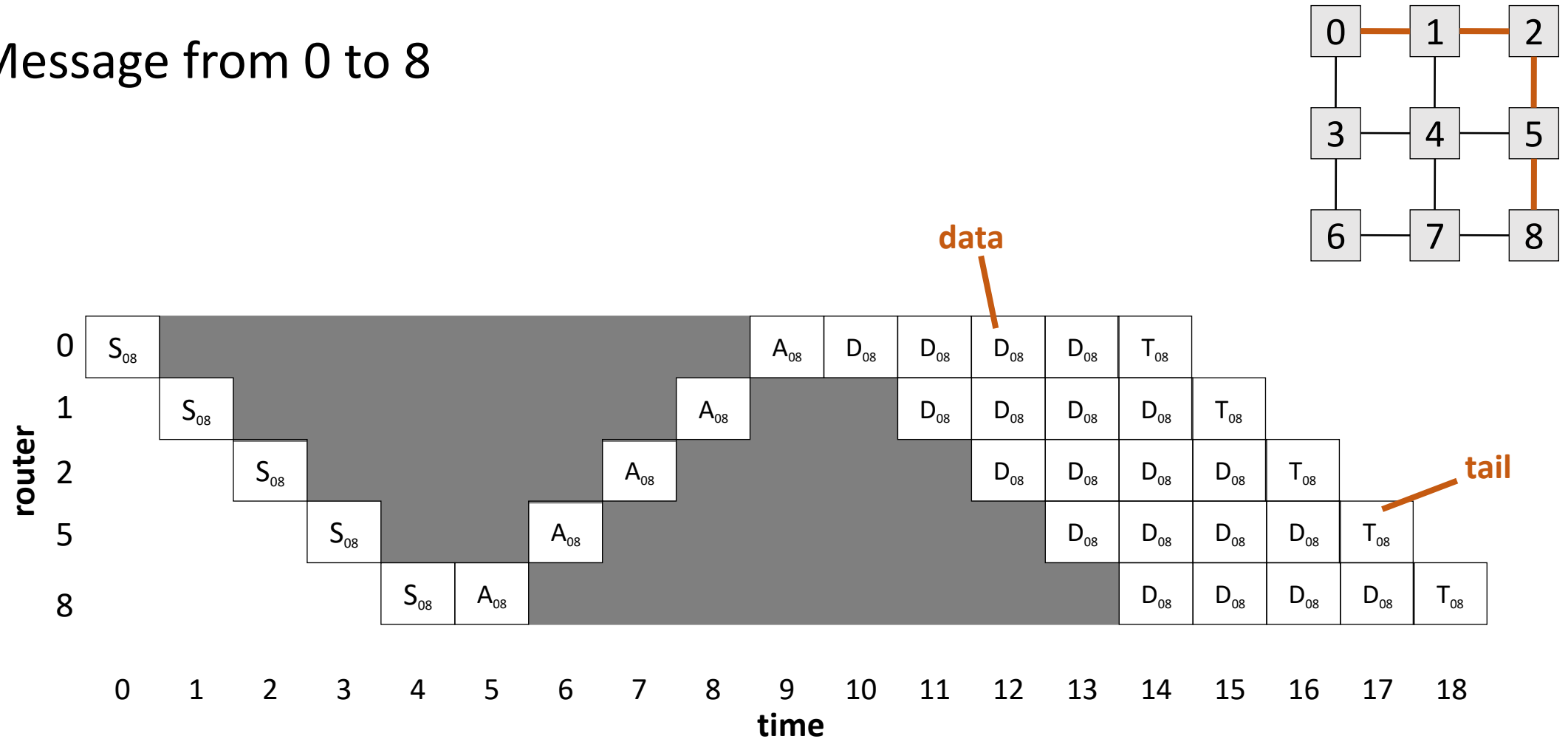
Circuit-Switching

e.g., Message from 0 to 8



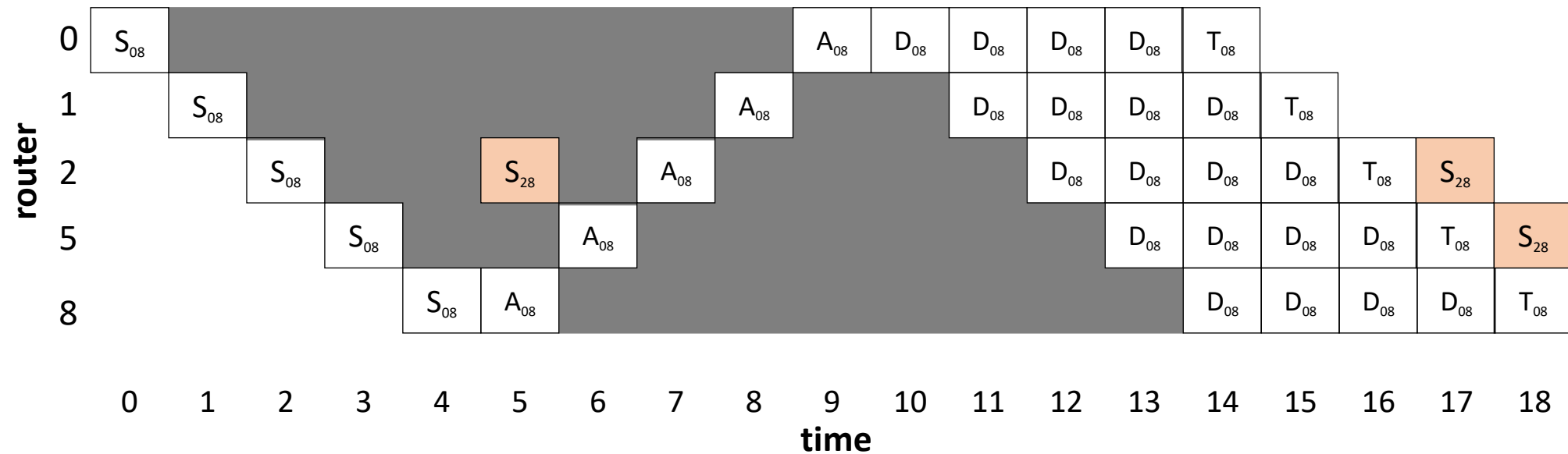
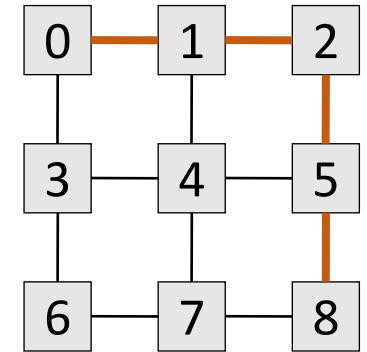
Circuit-Switching

e.g., Message from 0 to 8



Circuit-Switching

e.g., Message from 0 to 8, **and message from 2 to 8**



Circuit-Switching

e.g., Message from 0 to 8, **and message from 2 to 8**

